

Getting Started with the Kernel

In this chapter, we introduce some of the basics of the Linux kernel: where to get its source, how to compile it, and how to install the new kernel. We then go over the differences between the kernel and user-space programs and common programming constructs used in the kernel. Although the kernel certainly is unique in many ways, at the end of the day it is little different from any other large software project.

Obtaining the Kernel Source

The current Linux source code is always available in both a complete *tarball* (an archive created with the *tar* command) and an incremental patch from the official home of the Linux kernel, <http://www.kernel.org>.

Unless you have a specific reason to work with an older version of the Linux source, you *always* want the latest code. The repository at kernel.org is the place to get it, along with additional patches from a number of leading kernel developers.

Using Git

Over the last couple of years, the kernel hackers, led by Linus himself, have begun using a new version control system to manage the Linux kernel source. Linus created this system, called *Git*, with speed in mind. Unlike traditional systems such as *CVS*, *Git* is distributed, and its usage and workflow is consequently unfamiliar to many developers. I strongly recommend using *Git* to download and manage the Linux kernel source.

You can use *Git* to obtain a copy of the latest “pushed” version of Linus’s tree:

```
$ git clone git://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux-2.6.git
```

When checked out, you can update your tree to Linus’s latest:

```
$ git pull
```

With these two commands, you can obtain and subsequently keep up to date with the official kernel tree. To commit and manage your own changes, see Chapter 20, “Patches,

Hacking, and the Community.” A complete discussion of Git is outside the scope of this book; many online resources provide excellent guides.

Installing the Kernel Source

The kernel tarball is distributed in both GNU zip (gzip) and bzip2 format. Bzip2 is the default and preferred format because it generally compresses quite a bit better than gzip. The Linux kernel tarball in bzip2 format is named `linux-x.y.z.tar.bz2`, where *x.y.z* is the version of that particular release of the kernel source. After downloading the source, uncompressing and untarring it is simple. If your tarball is compressed with bzip2, run

```
$ tar xvjf linux-x.y.z.tar.bz2
```

If it is compressed with GNU zip, run

```
$ tar xvzf linux-x.y.z.tar.gz
```

This uncompresses and untars the source to the directory `linux-x.y.z`. If you use git to obtain and manage the kernel source, you do not need to download the tarball. Just run the *git clone* command as described and git downloads and unpacks the latest source.

Where to Install and Hack on the Source

The kernel source is typically installed in `/usr/src/linux`. You should not use this source tree for development because the kernel version against which your C library is compiled is often linked to this tree. Moreover, you should not require root in order to make changes to the kernel—instead, work out of your home directory and use root only to install new kernels. Even when installing a new kernel, `/usr/src/linux` should remain untouched.

Using Patches

Throughout the Linux kernel community, patches are the *lingua franca* of communication. You will distribute your code changes in patches and receive code from others as patches. *Incremental patches* provide an easy way to move from one kernel tree to the next. Instead of downloading each large tarball of the kernel source, you can simply apply an incremental patch to go from one version to the next. This saves everyone bandwidth and you time. To apply an incremental patch, from *inside* your kernel source tree, simply run

```
$ patch -p1 < ../patch-x.y.z
```

Generally, a patch to a given version of the kernel is applied against the previous version. Generating and applying patches is discussed in much more depth in later chapters.

The Kernel Source Tree

The kernel source tree is divided into a number of directories, most of which contain many more subdirectories. The directories in the root of the source tree, along with their descriptions, are listed in Table 2.1.

Table 2.1 Directories in the Root of the Kernel Source Tree

Directory	Description
arch	Architecture-specific source
block	Block I/O layer
crypto	Crypto API
Documentation	Kernel source documentation
drivers	Device drivers
firmware	Device firmware needed to use certain drivers
fs	The VFS and the individual filesystems
include	Kernel headers
init	Kernel boot and initialization
ipc	Interprocess communication code
kernel	Core subsystems, such as the scheduler
lib	Helper routines
mm	Memory management subsystem and the VM
net	Networking subsystem
samples	Sample, demonstrative code
scripts	Scripts used to build the kernel
security	Linux Security Module
sound	Sound subsystem
usr	Early user-space code (called initramfs)
tools	Tools helpful for developing Linux
virt	Virtualization infrastructure

A number of files in the root of the source tree deserve mention. The file `COPYING` is the kernel license (the GNU GPL v2). `CREDITS` is a listing of developers with more than a trivial amount of code in the kernel. `MAINTAINERS` lists the names of the individuals who maintain subsystems and drivers in the kernel. `Makefile` is the base kernel Makefile.

Building the Kernel

Building the kernel is easy. It is surprisingly easier than compiling and installing other system-level components, such as glibc. The 2.6 kernel series introduced a new configuration and build system, which made the job even easier and is a welcome improvement over earlier releases.

Configuring the Kernel

Because the Linux source code is available, it follows that you can configure and custom tailor it before compiling. Indeed, it is possible to compile support into your kernel for only the specific features and drivers you want. Configuring the kernel is a required step before building it. Because the kernel offers myriad features and supports a varied basket of hardware, there is a *lot* to configure. Kernel configuration is controlled by configuration options, which are prefixed by `CONFIG` in the form `CONFIG_FEATURE`. For example, symmetrical multiprocessing (SMP) is controlled by the configuration option `CONFIG_SMP`. If this option is set, SMP is enabled; if unset, SMP is disabled. The configure options are used both to decide which files to build and to manipulate code via preprocessor directives.

Configuration options that control the build process are either *Booleans* or *tristates*. A Boolean option is either *yes* or *no*. Kernel features, such as `CONFIG_PREEMPT`, are usually Booleans. A tristate option is one of *yes*, *no*, or *module*. The *module* setting represents a configuration option that is set but is to be compiled as a module (that is, a separate dynamically loadable object). In the case of tristates, a *yes* option explicitly means to compile the code into the main kernel image and not as a module. Drivers are usually represented by tristates.

Configuration options can also be strings or integers. These options do not control the build process but instead specify values that kernel source can access as a preprocessor macro. For example, a configuration option can specify the size of a statically allocated array.

Vendor kernels, such as those provided by Canonical for Ubuntu or Red Hat for Fedora, are precompiled as part of the distribution. Such kernels typically enable a good cross section of the needed kernel features and compile nearly all the drivers as modules. This provides for a great base kernel with support for a wide range of hardware as separate modules. For better or worse, as a kernel hacker, you need to compile your own kernels and learn what modules to include on your own.

Thankfully, the kernel provides multiple tools to facilitate configuration. The simplest tool is a text-based command-line utility:

```
$ make config
```

This utility goes through each option, one by one, and asks the user to interactively select *yes*, *no*, or (for tristates) *module*. Because this takes a *long* time, unless you are paid by the hour, you should use an ncurses-based graphical utility:

```
$ make menuconfig
```

Or a gtk+-based graphical utility:

```
$ make gconfig
```

These three utilities divide the various configuration options into categories, such as “Processor Type and Features.” You can move through the categories, view the kernel options, and of course change their values.

This command creates a configuration based on the defaults for your architecture:

```
$ make defconfig
```

Although these defaults are somewhat arbitrary (on i386, they are rumored to be Linus's configuration!), they provide a good start if you have never configured the kernel. To get off and running quickly, run this command and then go back and ensure that configuration options for your hardware are enabled.

The configuration options are stored in the root of the kernel source tree in a file named `.config`. You may find it easier (as most of the kernel developers do) to just edit this file directly. It is quite easy to search for and change the value of the configuration options. After making changes to your configuration file, or when using an existing configuration file on a new kernel tree, you can validate and update the configuration:

```
$ make oldconfig
```

You should always run this before building a kernel.

The configuration option `CONFIG_IKCONFIG_PROC` places the complete kernel configuration file, compressed, at `/proc/config.gz`. This makes it easy to clone your current configuration when building a new kernel. If your current kernel has this option enabled, you can copy the configuration out of `/proc` and use it to build a new kernel:

```
$ zcat /proc/config.gz > .config
$ make oldconfig
```

After the kernel configuration is set—however you do it—you can build it with a single command:

```
$ make
```

Unlike kernels before 2.6, you no longer need to run `make dep` before building the kernel—the dependency tree is maintained automatically. You also do not need to specify a specific build type, such as `bzImage`, or build modules separately, as you did in old versions. The default Makefile rule will handle everything.

Minimizing Build Noise

A trick to minimize build noise, but still see warnings and errors, is to redirect the output from `make`:

```
$ make > ../detritus
```

If you need to see the build output, you can read the file. Because the warnings and errors are output to standard error, however, you normally do not need to. In fact, I just do

```
$ make > /dev/null
```

This redirects all the worthless output to that big, ominous sink of no return, `/dev/null`.

Spawning Multiple Build Jobs

The `make` program provides a feature to split the build process into a number of parallel *jobs*. Each of these jobs then runs separately and concurrently, significantly speeding up the build process on multiprocessing systems. It also improves processor utilization because the time to build a large source tree includes significant time in I/O wait (time in which the process is idle waiting for an I/O request to complete).

By default, `make` spawns only a single job because Makefiles all too often have incorrect dependency information. With incorrect dependencies, multiple jobs can step on each other's toes, resulting in errors in the build process. The kernel's Makefiles have correct dependency information, so spawning multiple jobs does not result in failures. To build the kernel with multiple `make` jobs, use

```
$ make -jn
```

Here, *n* is the number of jobs to spawn. Usual practice is to spawn one or two jobs per processor. For example, on a 16-core machine, you might do

```
$ make -j32 > /dev/null
```

Using utilities such as the excellent `distcc` or `ccache` can also dramatically improve kernel build time.

Installing the New Kernel

After the kernel is built, you need to install it. How it is installed is architecture- and boot loader-dependent—consult the directions for your boot loader on where to copy the kernel image and how to set it up to boot. Always keep a known-safe kernel or two around in case your new kernel has problems!

As an example, on an x86 system using `grub`, you would copy `arch/i386/boot/bzImage` to `/boot`, name it something like `vmlinuz-version`, and edit `/boot/grub/grub.conf`, adding a new entry for the new kernel. Systems using `LILO` to boot would instead edit `/etc/lilo.conf` and then rerun `lilo`.

Installing modules, thankfully, is automated and architecture-independent. As root, simply run

```
% make modules_install
```

This installs all the compiled modules to their correct home under `/lib/modules`.

The build process also creates the file `System.map` in the root of the kernel source tree. It contains a symbol lookup table, mapping kernel symbols to their start addresses. This is used during debugging to translate memory addresses to function and variable names.

A Beast of a Different Nature

The Linux kernel has several unique attributes as compared to a normal user-space application. Although these differences do not necessarily make developing kernel code *harder* than developing user-space code, they certainly make doing so *different*.

These characteristics make the kernel a beast of a different nature. Some of the usual rules are bent; other rules are entirely new. Although some of the differences are obvious (we all know the kernel can do anything it wants), others are not so obvious. The most important of these differences are

- The kernel has access to neither the C library nor the standard C headers.
- The kernel is coded in GNU C.
- The kernel lacks the memory protection afforded to user-space.
- The kernel cannot easily execute floating-point operations.
- The kernel has a small per-process fixed-size stack.
- Because the kernel has asynchronous interrupts, is preemptive, and supports SMP, synchronization and concurrency are major concerns within the kernel.
- Portability is important.

Let's briefly look at each of these issues because all kernel developers must keep them in mind.

No libc or Standard Headers

Unlike a user-space application, the kernel is not linked against the standard C library—or any other library, for that matter. There are multiple reasons for this, including a chicken-and-the-egg situation, but the primary reason is speed and size. The full C library—or even a decent subset of it—is too large and too inefficient for the kernel.

Do not fret: Many of the usual libc functions are implemented inside the kernel. For example, the common string manipulation functions are in `lib/string.c`. Just include the header file `<linux/string.h>` and have at them.

Header Files

When I talk about header files in this book, I am referring to the kernel header files that are part of the kernel source tree. Kernel source files cannot include outside headers, just as they cannot use outside libraries.

The base files are located in the `include/` directory in the root of the kernel source tree. For example, the header file `<linux/inotify.h>` is located at `include/linux/inotify.h` in the kernel source tree.

A set of architecture-specific header files are located in `arch/<architecture>/include/asm` in the kernel source tree. For example, if compiling for the x86 architecture, your architecture-specific headers are in `arch/x86/include/asm`. Source code includes these headers via just the `asm/` prefix, for example `<asm/ioctl.h>`.

Of the missing functions, the most familiar is `printf()`. The kernel does not have access to `printf()`, but it does provide `printk()`, which works pretty much the same as its more familiar cousin. The `printk()` function copies the formatted string into the kernel log buffer, which is normally read by the `syslog` program. Usage is similar to `printf()`:

```
printk("Hello world! A string '%s' and an integer '%d'\n", str, i);
```

One notable difference between `printf()` and `printk()` is that `printk()` enables you to specify a priority flag. This flag is used by `syslogd` to decide where to display kernel messages. Here is an example of these priorities:

```
printk(KERN_ERR "this is an error!\n");
```

Note there is no comma between `KERN_ERR` and the printed message. This is intentional; the priority flag is a preprocessor-define representing a string literal, which is concatenated onto the printed message during compilation. We use `printk()` throughout this book.

GNU C

Like any self-respecting Unix kernel, the Linux kernel is programmed in C. Perhaps surprisingly, the kernel is not programmed in strict ANSI C. Instead, where applicable, the kernel developers make use of various language extensions available in *gcc* (the GNU Compiler Collection, which contains the C compiler used to compile the kernel and most everything else written in C on a Linux system).

The kernel developers use both ISO C99¹ and GNU C extensions to the C language. These changes wed the Linux kernel to *gcc*, although recently one other compiler, the Intel C compiler, has sufficiently supported enough *gcc* features that it, too, can compile the Linux kernel. The earliest supported *gcc* version is 3.2; *gcc* version 4.4 or later is recommended. The ISO C99 extensions that the kernel uses are nothing special and, because C99 is an official revision of the C language, are slowly cropping up in a lot of other code. The more unfamiliar deviations from standard ANSI C are those provided by GNU C. Let's look at some of the more interesting extensions that you will see in the kernel; these changes differentiate kernel code from other projects with which you might be familiar.

Inline Functions

Both C99 and GNU C support *inline functions*. An inline function is, as its name suggests, inserted inline into each function call site. This eliminates the overhead of function invocation and return (register saving and restore) and allows for potentially greater optimization as the compiler can optimize both the caller and the called function as one. As a downside (nothing in life is free), code size increases because the contents of the function are copied into all the callers, which increases memory consumption and instruction cache footprint. Kernel developers use inline functions for small time-critical functions.

¹ ISO C99 is the latest major revision to the ISO C standard. C99 adds numerous enhancements to the previous major revision, ISO C90, including designated initializers, variable length arrays, C++-style comments, and the *long long* and *complex* types. The Linux kernel, however, employs only a subset of C99 features.

Making large functions inline, especially those used more than once or that are not exceedingly time critical, is frowned upon.

An inline function is declared when the keywords `static` and `inline` are used as part of the function definition. For example

```
static inline void wolf(unsigned long tail_size)
```

The function declaration must precede any usage, or else the compiler cannot make the function inline. Common practice is to place inline functions in header files. Because they are marked `static`, an exported function is not created. If an inline function is used by only one file, it can instead be placed toward the top of just that file.

In the kernel, using inline functions is preferred over complicated macros for reasons of type safety and readability.

Inline Assembly

The gcc C compiler enables the embedding of assembly instructions in otherwise normal C functions. This feature, of course, is used in only those parts of the kernel that are unique to a given system architecture.

The `asm()` compiler directive is used to inline assembly code. For example, this inline assembly directive executes the x86 processor's `rdtsc` instruction, which returns the value of the timestamp (`tsc`) register:

```
unsigned int low, high;
asm volatile("rdtsc" : "=a" (low), "=d" (high));
/* low and high now contain the lower and upper 32-bits of the 64-bit tsc */
```

The Linux kernel is written in a mixture of C and assembly, with assembly relegated to low-level architecture and fast path code. The vast majority of kernel code is programmed in straight C.

Branch Annotation

The gcc C compiler has a built-in directive that optimizes conditional branches as either very likely taken or very unlikely taken. The compiler uses the directive to appropriately optimize the branch. The kernel wraps the directive in easy-to-use macros, `likely()` and `unlikely()`.

For example, consider an `if` statement such as the following:

```
if (error) {
    /* ... */
}
```

To mark this branch as very unlikely taken (that is, likely not taken):

```
/* we predict 'error' is nearly always zero ... */
if (unlikely(error)) {
    /* ... */
}
```

Conversely, to mark a branch as very likely taken:

```
/* we predict 'success' is nearly always nonzero ... */
if (likely(success)) {
    /* ... */
}
```

You should only use these directives when the branch direction is overwhelmingly known *a priori* or when you want to optimize a specific case at the cost of the other case. This is an important point: These directives result in a performance boost when the branch is correctly marked, but a performance *loss* when the branch is mismarked. A common usage, as shown in these examples, for `unlikely()` and `likely()` is error conditions. As you might expect, `unlikely()` finds much more use in the kernel because `if` statements tend to indicate a special case.

No Memory Protection

When a user-space application attempts an illegal memory access, the kernel can trap the error, send the `SIGSEGV` signal, and kill the process. If the kernel attempts an illegal memory access, however, the results are less controlled. (After all, who is going to look after the kernel?) Memory violations in the kernel result in an *oops*, which is a major kernel error. It should go without saying that you must not illegally access memory, such as dereferencing a `NULL` pointer—but within the kernel, the stakes are much higher!

Additionally, kernel memory is not pageable. Therefore, every byte of memory you consume is one less byte of available physical memory. Keep that in mind the next time you need to add *one more feature* to the kernel!

No (Easy) Use of Floating Point

When a user-space process uses floating-point instructions, the kernel manages the transition from integer to floating point mode. What the kernel has to do when using floating-point instructions varies by architecture, but the kernel normally catches a trap and then initiates the transition from integer to floating point mode.

Unlike user-space, the kernel does not have the luxury of seamless support for floating point because it cannot easily trap itself. Using a floating point inside the kernel requires manually saving and restoring the floating point registers, among other possible chores. The short answer is: *Don't do it!* Except in the rare cases, no floating-point operations are in the kernel.

Small, Fixed-Size Stack

User-space can get away with statically allocating many variables on the stack, including huge structures and thousand-element arrays. This behavior is legal because user-space has a large stack that can dynamically grow. (Developers on older, less advanced operating systems—say, DOS—might recall a time when even user-space had a fixed-sized stack.)

The kernel stack is neither large nor dynamic; it is small and fixed in size. The exact size of the kernel's stack varies by architecture. On x86, the stack size is configurable at compile-time and can be either 4KB or 8KB. Historically, the kernel stack is two pages, which generally implies that it is 8KB on 32-bit architectures and 16KB on 64-bit architectures—this size is fixed and absolute. Each process receives its own stack.

The kernel stack is discussed in much greater detail in later chapters.

Synchronization and Concurrency

The kernel is susceptible to race conditions. Unlike a single-threaded user-space application, a number of properties of the kernel allow for concurrent access of shared resources and thus require synchronization to prevent races. Specifically

- Linux is a preemptive multitasking operating system. Processes are scheduled and rescheduled at the whim of the kernel's process scheduler. The kernel must synchronize between these tasks.
- Linux supports symmetrical multiprocessing (SMP). Therefore, without proper protection, kernel code executing simultaneously on two or more processors can concurrently access the same resource.
- Interrupts occur asynchronously with respect to the currently executing code. Therefore, without proper protection, an interrupt can occur in the midst of accessing a resource, and the interrupt handler can then access the same resource.
- The Linux kernel is preemptive. Therefore, without protection, kernel code can be preempted in favor of different code that then accesses the same resource.

Typical solutions to race conditions include spinlocks and semaphores. Later chapters provide a thorough discussion of synchronization and concurrency.

Importance of Portability

Although user-space applications do not *have* to aim for portability, Linux is a portable operating system and should remain one. This means that architecture-independent C code must correctly compile and run on a wide range of systems, and that architecture-dependent code must be properly segregated in system-specific directories in the kernel source tree.

A handful of rules—such as remain endian neutral, be 64-bit clean, do not assume the word or page size, and so on—go a long way. Portability is discussed in depth in a later chapter.

Conclusion

To be sure, the kernel has unique qualities. It enforces its own rules and the stakes, managing the entire system as the kernel does, are certainly higher. That said, the Linux kernel's complexity and barrier-to-entry is not qualitatively different from any other large soft-

ware project. The most important step on the road to Linux development is the realization that the kernel is not something to fear. Unfamiliar, sure. Insurmountable? Not at all.

This and the previous chapter lay the foundation for the topics we cover through this book's remaining chapters. In each subsequent chapter, we cover a specific kernel concept or subsystem. Along the way, it is imperative that you read and modify the kernel source. Only through actually reading and experimenting with the code can you ever understand it. The source is freely available—use it!

Process Management

This chapter introduces the concept of the *process*, one of the fundamental abstractions in Unix operating systems. It defines the process, as well as related concepts such as threads, and then discusses how the Linux kernel manages each process: how they are enumerated within the kernel, how they are created, and how they ultimately die. Because running user applications is the reason we have operating systems, the process management is a crucial part of any operating system kernel, including Linux.

The Process

A *process* is a program (object code stored on some media) in the midst of execution. Processes are, however, more than just the executing program code (often called the *text section* in Unix). They also include a set of resources such as open files and pending signals, internal kernel data, processor state, a memory address space with one or more memory mappings, one or more *threads of execution*, and a *data section* containing global variables. Processes, in effect, are the living result of running program code. The kernel needs to manage all these details efficiently and transparently.

Threads of execution, often shortened to *threads*, are the objects of activity within the process. Each thread includes a unique program counter, process stack, and set of processor registers. The kernel schedules individual threads, not processes. In traditional Unix systems, each process consists of one thread. In modern systems, however, multithreaded programs—those that consist of more than one thread—are common. As you will see later, Linux has a unique implementation of threads: It does not differentiate between threads and processes. To Linux, a thread is just a special kind of process.

On modern operating systems, processes provide two virtualizations: a virtualized processor and virtual memory. The virtual processor gives the process the illusion that it alone monopolizes the system, despite possibly sharing the processor among hundreds of other processes. Chapter 4, “Process Scheduling,” discusses this virtualization. Virtual memory lets the process allocate and manage memory as if it alone owned all the memory in the system. Virtual memory is covered in Chapter 12, “Memory Management.”

Interestingly, note that threads share the virtual memory abstraction, whereas each receives its own virtualized processor.

A program itself is not a process; a process is an *active* program and related resources. Indeed, two or more processes can exist that are executing the *same* program. In fact, two or more processes can exist that share various resources, such as open files or an address space.

A process begins its life when, not surprisingly, it is created. In Linux, this occurs by means of the `fork()` system call, which creates a new process by duplicating an existing one. The process that calls `fork()` is the *parent*, whereas the new process is the *child*. The parent resumes execution and the child starts execution at the same place: where the call to `fork()` returns. The `fork()` system call returns from the kernel twice: once in the parent process and again in the newborn child.

Often, immediately after a fork it is desirable to execute a new, different program. The `exec()` family of function calls creates a new address space and loads a new program into it. In contemporary Linux kernels, `fork()` is actually implemented via the `clone()` system call, which is discussed in a following section.

Finally, a program exits via the `exit()` system call. This function terminates the process and frees all its resources. A parent process can inquire about the status of a terminated child via the `wait4()`¹ system call, which enables a process to wait for the termination of a specific process. When a process exits, it is placed into a special zombie state that represents terminated processes until the parent calls `wait()` or `waitpid()`.

Note

Another name for a process is a *task*. The Linux kernel internally refers to processes as tasks. In this book, I use the terms interchangeably, although when I say task I am generally referring to a process from the kernel's point of view.

Process Descriptor and the Task Structure

The kernel stores the list of processes in a circular doubly linked list called the *task list*.² Each element in the task list is a *process descriptor* of the type `struct task_struct`, which is defined in `<linux/sched.h>`. The process descriptor contains all the information about a specific process.

The `task_struct` is a relatively large data structure, at around 1.7 kilobytes on a 32-bit machine. This size, however, is quite small considering that the structure contains all the information that the kernel has and needs about a process. The process descriptor contains

¹ The kernel implements the `wait4()` system call. Linux systems, via the C library, typically provide the `wait()`, `waitpid()`, `wait3()`, and `wait4()` functions. All these functions return status about a terminated process, albeit with slightly different semantics.

² Some texts on operating system design call this list the task array. Because the Linux implementation is a linked list and not a static array, in Linux it is called the task list.

the data that describes the executing program—open files, the process’s address space, pending signals, the process’s state, and much more (see Figure 3.1).

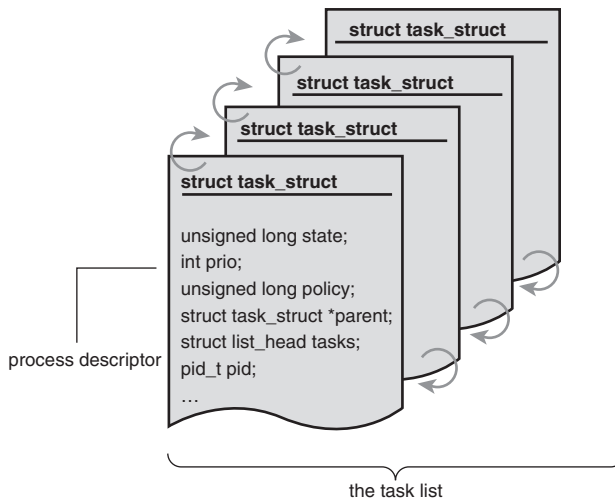


Figure 3.1 The process descriptor and task list.

Allocating the Process Descriptor

The `task_struct` structure is allocated via the *slab allocator* to provide object reuse and cache coloring (see Chapter 12). Prior to the 2.6 kernel series, `struct task_struct` was stored at the end of the kernel stack of each process. This allowed architectures with few registers, such as x86, to calculate the location of the process descriptor via the *stack pointer* without using an extra register to store the location. With the process descriptor now dynamically created via the slab allocator, a new structure, `struct thread_info`, was created that again lives at the bottom of the stack (for stacks that grow down) and at the top of the stack (for stacks that grow up).³ See Figure 3.2.

The `thread_info` structure is defined on x86 in `<asm/thread_info.h>` as

```
struct thread_info {
    struct task_struct    *task;
    struct exec_domain    *exec_domain;
    __u32                 flags;
    __u32                 status;
    __u32                 cpu;
    int                   preempt_count;
```

³ Register-impaired architectures were not the only reason for creating `struct thread_info`. The new structure also makes it rather easy to calculate offsets of its values for use in assembly code.

```

mm_segment_t      addr_limit;
struct restart_block restart_block;
void              *sysenter_return;
int               uaccess_err;
};

```

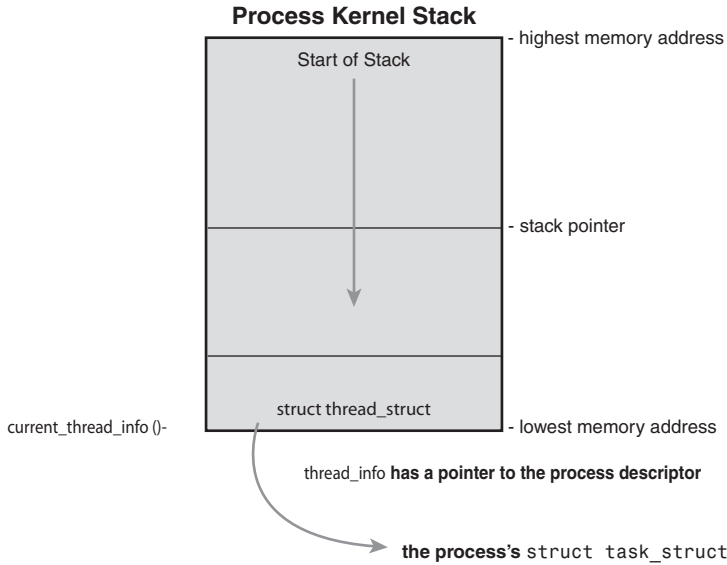


Figure 3.2 The process descriptor and kernel stack.

Each task's `thread_info` structure is allocated at the end of its stack. The `task` element of the structure is a pointer to the task's actual `task_struct`.

Storing the Process Descriptor

The system identifies processes by a unique *process identification* value or *PID*. The PID is a numerical value represented by the opaque type⁴ `pid_t`, which is typically an `int`. Because of backward compatibility with earlier Unix and Linux versions, however, the default maximum value is only 32,768 (that of a `short int`), although the value optionally can be increased as high as four million (this is controlled in `<linux/threads.h>`). The kernel stores this value as `pid` inside each process descriptor.

This maximum value is important because it is essentially the maximum number of processes that may exist concurrently on the system. Although 32,768 might be sufficient for a desktop system, large servers may require many more processes. Moreover, the lower the value, the sooner the values will wrap around, destroying the useful notion that higher

⁴ An opaque type is a data type whose physical representation is unknown or irrelevant.

values indicate later-run processes than lower values. If the system is willing to break compatibility with old applications, the administrator may increase the maximum value via `/proc/sys/kernel/pid_max`.

Inside the kernel, tasks are typically referenced directly by a pointer to their `task_struct` structure. In fact, most kernel code that deals with processes works directly with `struct task_struct`. Consequently, it is useful to be able to quickly look up the process descriptor of the currently executing task, which is done via the `current` macro. This macro must be independently implemented by each architecture. Some architectures save a pointer to the `task_struct` structure of the currently running process in a register, enabling for efficient access. Other architectures, such as x86 (which has few registers to waste), make use of the fact that `struct thread_info` is stored on the kernel stack to calculate the location of `thread_info` and subsequently the `task_struct`.

On x86, `current` is calculated by masking out the 13 least-significant bits of the stack pointer to obtain the `thread_info` structure. This is done by the `current_thread_info()` function. The assembly is shown here:

```
movl $-8192, %eax
andl %esp, %eax
```

This assumes that the stack size is 8KB. When 4KB stacks are enabled, 4096 is used in lieu of 8192.

Finally, `current` dereferences the `task` member of `thread_info` to return the `task_struct`:

```
current_thread_info()->task;
```

Contrast this approach with that taken by PowerPC (IBM's modern RISC-based microprocessor), which stores the current `task_struct` in a register. Thus, `current` on PPC merely returns the value stored in the register `r2`. PPC can take this approach because, unlike x86, it has plenty of registers. Because accessing the process descriptor is a common and important job, the PPC kernel developers deem using a register worthy for the task.

Process State

The `state` field of the process descriptor describes the current condition of the process (see Figure 3.3). Each process on the system is in exactly one of five different states. This value is represented by one of five flags:

- **TASK_RUNNING**—The process is runnable; it is either currently running or on a run-queue waiting to run (run-queues are discussed in Chapter 4). This is the only possible state for a process executing in user-space; it can also apply to a process in kernel-space that is actively running.
- **TASK_INTERRUPTIBLE**—The process is sleeping (that is, it is blocked), waiting for some condition to exist. When this condition exists, the kernel sets the process's state to **TASK_RUNNING**. The process also awakes prematurely and becomes runnable if it receives a signal.

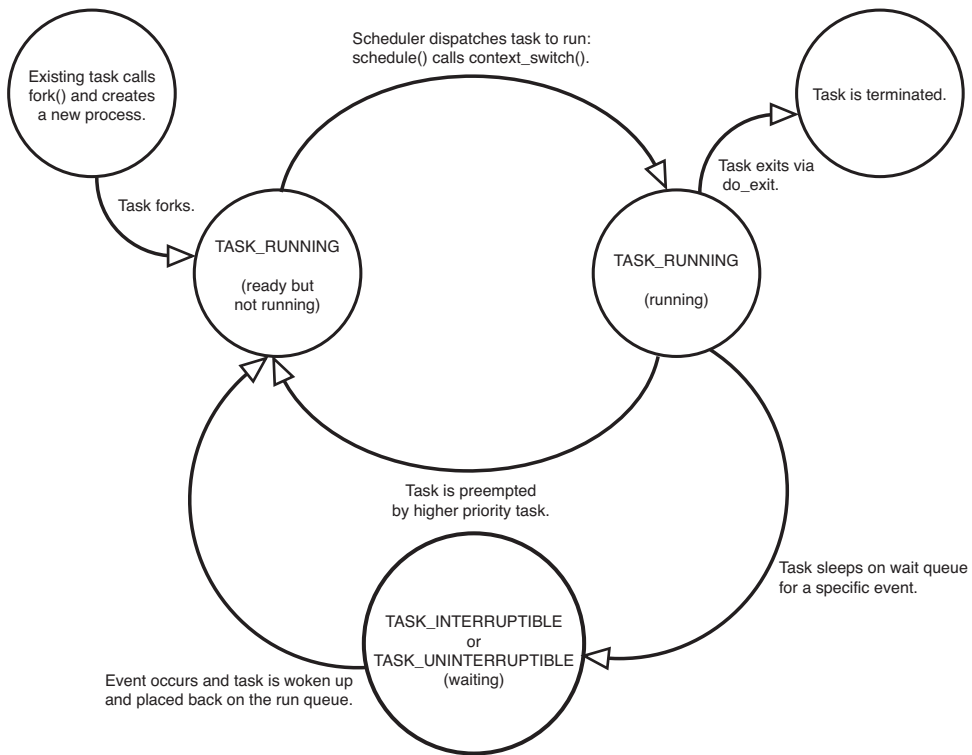


Figure 3.3 Flow chart of process states.

- `TASK_UNINTERRUPTIBLE`—This state is identical to `TASK_INTERRUPTIBLE` except that it does *not* wake up and become runnable if it receives a signal. This is used in situations where the process must wait without interruption or when the event is expected to occur quite quickly. Because the task does not respond to signals in this state, `TASK_UNINTERRUPTIBLE` is less often used than `TASK_INTERRUPTIBLE`.⁵
- `__TASK_TRACED`—The process is being *traced* by another process, such as a debugger, via `ptrace`.
- `__TASK_STOPPED`—Process execution has stopped; the task is not running nor is it eligible to run. This occurs if the task receives the `SIGSTOP`, `SIGTSTP`, `SIGTTIN`, or `SIGTTOU` signal or if it receives *any* signal while it is being debugged.

⁵ This is why you have those dreaded unkillable processes with state `D` in `ps (1)`. Because the task will not respond to signals, you cannot send it a `SIGKILL` signal. Further, even if you could terminate the task, it would not be wise because the task is supposedly in the middle of an important operation and may hold a semaphore.

Manipulating the Current Process State

Kernel code often needs to change a process's state. The preferred mechanism is using

```
set_task_state(task, state);          /* set task 'task' to state 'state' */
```

This function sets the given task to the given state. If applicable, it also provides a memory barrier to force ordering on other processors. (This is only needed on SMP systems.) Otherwise, it is equivalent to

```
task->state = state;
```

The method `set_current_state(state)` is synonymous to `set_task_state(current, state)`. See `<linux/sched.h>` for the implementation of these and related functions.

Process Context

One of the most important parts of a process is the executing program code. This code is read in from an *executable file* and executed within the program's address space. Normal program execution occurs in *user-space*. When a program executes a system call (see Chapter 5, "System Calls") or triggers an exception, it enters *kernel-space*. At this point, the kernel is said to be "executing on behalf of the process" and is in *process context*. When in process context, the `current` macro is valid.⁶ Upon exiting the kernel, the process resumes execution in user-space, unless a higher-priority process has become runnable in the interim, in which case the scheduler is invoked to select the higher priority process.

System calls and exception handlers are well-defined interfaces into the kernel. A process can begin executing in kernel-space only through one of these interfaces—all access to the kernel is through these interfaces.

The Process Family Tree

A distinct hierarchy exists between processes in Unix systems, and Linux is no exception. All processes are descendants of the `init` process, whose PID is one. The kernel starts `init` in the last step of the boot process. The `init` process, in turn, reads the system *initscripts* and executes more programs, eventually completing the boot process.

Every process on the system has exactly one parent. Likewise, every process has zero or more children. Processes that are all direct children of the same parent are called *siblings*. The relationship between processes is stored in the process descriptor. Each `task_struct` has a pointer to the parent's `task_struct`, named `parent`, and a list of children, named

⁶ Other than process context there is interrupt context, which we discuss in Chapter 7, "Interrupts and Interrupt Handlers." In interrupt context, the system is not running on behalf of a process but is executing an interrupt handler. No process is tied to interrupt handlers.

children. Consequently, given the current process, it is possible to obtain the process descriptor of its parent with the following code:

```
struct task_struct *my_parent = current->parent;
```

Similarly, it is possible to iterate over a process's children with

```
struct task_struct *task;
struct list_head *list;

list_for_each(list, &current->children) {
    task = list_entry(list, struct task_struct, sibling);
    /* task now points to one of current's children */
}
```

The init task's process descriptor is statically allocated as `init_task`. A good example of the relationship between all processes is the fact that this code will always succeed:

```
struct task_struct *task;

for (task = current; task != &init_task; task = task->parent)
    ;
/* task now points to init */
```

In fact, you can follow the process hierarchy from any one process in the system to *any* other. Oftentimes, however, it is desirable simply to iterate over *all* processes in the system. This is easy because the task list is a circular, doubly linked list. To obtain the next task in the list, given any valid task, use

```
list_entry(task->tasks.next, struct task_struct, tasks)
```

Obtaining the previous task works the same way:

```
list_entry(task->tasks.prev, struct task_struct, tasks)
```

These two routines are provided by the macros `next_task(task)` and `prev_task(task)`, respectively. Finally, the macro `for_each_process(task)` is provided, which iterates over the entire task list. On each iteration, `task` points to the next task in the list:

```
struct task_struct *task;

for_each_process(task) {
    /* this pointlessly prints the name and PID of each task */
    printk("%s[%d]\n", task->comm, task->pid);
}
```

Caution

It is expensive to iterate over every task in a system with many processes; code should have good reason (and no alternative) before doing so.

Process Creation

Process creation in Unix is unique. Most operating systems implement a *spawn* mechanism to create a new process in a new address space, read in an executable, and begin executing it. Unix takes the unusual approach of separating these steps into two distinct functions: `fork()` and `exec()`.⁷ The first, `fork()`, creates a child process that is a copy of the current task. It differs from the parent only in its PID (which is unique), its PPID (parent's PID, which is set to the original process), and certain resources and statistics, such as pending signals, which are not inherited. The second function, `exec()`, loads a new executable into the address space and begins executing it. The combination of `fork()` followed by `exec()` is similar to the single function most operating systems provide.

Copy-on-Write

Traditionally, upon `fork()`, all resources owned by the parent are duplicated and the copy is given to the child. This approach is naive and inefficient in that it copies much data that might otherwise be shared. Worse still, if the new process were to immediately execute a new image, all that copying would go to waste. In Linux, `fork()` is implemented through the use of copy-on-write pages. Copy-on-write (or *COW*) is a technique to delay or altogether prevent copying of the data. Rather than duplicate the process address space, the parent and the child can share a single copy.

The data, however, is marked in such a way that if it is written to, a duplicate is made and each process receives a unique copy. Consequently, the duplication of resources occurs only when they are written; until then, they are shared read-only. This technique delays the copying of each page in the address space until it is actually written to. In the case that the pages are never written—for example, if `exec()` is called immediately after `fork()`—they never need to be copied.

The only overhead incurred by `fork()` is the duplication of the parent's page tables and the creation of a unique process descriptor for the child. In the common case that a process executes a new executable image immediately after forking, this optimization prevents the wasted copying of large amounts of data (with the address space, easily tens of megabytes). This is an important optimization because the Unix philosophy encourages quick process execution.

⁷ By `exec()` I mean any member of the `exec()` family of functions. The kernel implements the `execve()` system call on top of which `execip()`, `execle()`, `execv()`, and `execvp()` are implemented.

Forking

Linux implements `fork()` via the `clone()` system call. This call takes a series of flags that specify which resources, if any, the parent and child process should share. (See “The Linux Implementation of Threads” section later in this chapter for more about the flags.) The `fork()`, `vfork()`, and `__clone()` library calls all invoke the `clone()` system call with the requisite flags. The `clone()` system call, in turn, calls `do_fork()`.

The bulk of the work in forking is handled by `do_fork()`, which is defined in `kernel/fork.c`. This function calls `copy_process()` and then starts the process running. The interesting work is done by `copy_process()`:

1. It calls `dup_task_struct()`, which creates a new kernel stack, `thread_info` structure, and `task_struct` for the new process. The new values are identical to those of the current task. At this point, the child and parent process descriptors are identical.
2. It then checks that the new child will not exceed the resource limits on the number of processes for the current user.
3. The child needs to differentiate itself from its parent. Various members of the process descriptor are cleared or set to initial values. Members of the process descriptor not inherited are primarily statistically information. The bulk of the values in `task_struct` remain unchanged.
4. The child’s state is set to `TASK_UNINTERRUPTIBLE` to ensure that it does not yet run.
5. `copy_process()` calls `copy_flags()` to update the `flags` member of the `task_struct`. The `PF_SUPERPRIV` flag, which denotes whether a task used super-user privileges, is cleared. The `PF_FORKNOEXEC` flag, which denotes a process that has not called `exec()`, is set.
6. It calls `alloc_pid()` to assign an available PID to the new task.
7. Depending on the flags passed to `clone()`, `copy_process()` either duplicates or shares open files, filesystem information, signal handlers, process address space, and namespace. These resources are typically shared between threads in a given process; otherwise they are unique and thus copied here.
8. Finally, `copy_process()` cleans up and returns to the caller a pointer to the new child.

Back in `do_fork()`, if `copy_process()` returns successfully, the new child is woken up and run. Deliberately, the kernel runs the child process first.⁸ In the common case of the child simply calling `exec()` immediately, this eliminates any copy-on-write overhead that would occur if the parent ran first and began writing to the address space.

⁸ This does not currently function correctly, although the goal is for the child to run first.

vfork()

The `vfork()` system call has the same effect as `fork()`, except that the page table entries of the parent process are not copied. Instead, the child executes as the sole thread in the parent's address space, and the parent is blocked until the child either calls `exec()` or exits. The child is *not* allowed to write to the address space. This was a welcome optimization in the old days of 3BSD when the call was introduced because at the time copy-on-write pages were not used to implement `fork()`. Today, with copy-on-write and child-runs-first semantics, the only benefit to `vfork()` is not copying the parent page tables entries. If Linux one day gains copy-on-write page table entries, there will no longer be any benefit.⁹ Because the semantics of `vfork()` are tricky (what, for example, happens if the `exec()` fails?), ideally systems would not need `vfork()` and the kernel would not implement it. It is entirely possible to implement `vfork()` as a normal `fork()`—this is what Linux did until version 2.2.

The `vfork()` system call is implemented via a special flag to the `clone()` system call:

1. In `copy_process()`, the `task_struct` member `vfork_done` is set to `NULL`.
2. In `do_fork()`, if the special flag was given, `vfork_done` is pointed at a specific address.
3. After the child is first run, the parent—instead of returning—waits for the child to signal it through the `vfork_done` pointer.
4. In the `mm_release()` function, which is used when a task exits a memory address space, `vfork_done` is checked to see whether it is `NULL`. If it is not, the parent is signaled.
5. Back in `do_fork()`, the parent wakes up and returns.

If this all goes as planned, the child is now executing in a new address space, and the parent is again executing in its original address space. The overhead is lower, but the implementation is not pretty.

The Linux Implementation of Threads

Threads are a popular modern programming abstraction. They provide multiple threads of execution within the same program in a shared memory address space. They can also share open files and other resources. Threads enable *concurrent programming* and, on multiple processor systems, true *parallelism*.

Linux has a unique implementation of threads. To the Linux kernel, there is no concept of a thread. Linux implements all threads as standard processes. The Linux kernel

⁹ Patches are available to add this functionality to Linux. In time, this feature will most likely find its way into the mainline Linux kernel.

does not provide any special scheduling semantics or data structures to represent threads. Instead, a thread is merely a process that shares certain resources with other processes. Each thread has a unique `task_struct` and appears to the kernel as a normal process—threads just happen to share resources, such as an address space, with other processes.

This approach to threads contrasts greatly with operating systems such as Microsoft Windows or Sun Solaris, which have *explicit* kernel support for threads (and sometimes call threads *lightweight processes*). The name “lightweight process” sums up the difference in philosophies between Linux and other systems. To these other operating systems, threads are an abstraction to provide a lighter, quicker execution unit than the heavy process. To Linux, threads are simply a manner of sharing resources between processes (which are already quite lightweight).¹⁰ For example, assume you have a process that consists of four threads. On systems with explicit thread support, one process descriptor might exist that, in turn, points to the four different threads. The process descriptor describes the shared resources, such as an address space or open files. The threads then describe the resources they alone possess. Conversely, in Linux, there are simply four processes and thus four normal `task_struct` structures. The four processes are set up to share certain resources. The result is quite elegant.

Creating Threads

Threads are created the same as normal tasks, with the exception that the `clone()` system call is passed flags corresponding to the specific resources to be shared:

```
clone(CLONE_VM | CLONE_FS | CLONE_FILES | CLONE_SIGHAND, 0);
```

The previous code results in behavior identical to a normal `fork()`, except that the address space, filesystem resources, file descriptors, and signal handlers are shared. In other words, the new task and its parent are what are popularly called *threads*.

In contrast, a normal `fork()` can be implemented as

```
clone(SIGCHLD, 0);
```

And `vfork()` is implemented as

```
clone(CLONE_VFORK | CLONE_VM | SIGCHLD, 0);
```

The flags provided to `clone()` help specify the behavior of the new process and detail what resources the parent and child will share. Table 3.1 lists the clone flags, which are defined in `<linux/sched.h>`, and their effect.

¹⁰ As an example, benchmark process creation time in Linux versus process (or even thread!) creation time in these other operating systems. The results are favorable for Linux.

Flag	Meaning
CLONE_FILES	Parent and child share open files.
CLONE_FS	Parent and child share filesystem information.
CLONE_IDLETASK	Set PID to zero (used only by the idle tasks).
CLONE_NEWNS	Create a new namespace for the child.
CLONE_PARENT	Child is to have same parent as its parent.
CLONE_PTRACE	Continue tracing child.
CLONE_SETTID	Write the TID back to user-space.
CLONE_SETTLS	Create a new TLS for the child.
CLONE_SIGHAND	Parent and child share signal handlers and blocked signals.
CLONE_SYSVSEM	Parent and child share System V SEM_UNDO semantics.
CLONE_THREAD	Parent and child are in the same thread group.
CLONE_VFORK	<code>vfork()</code> was used and the parent will sleep until the child wakes it.
CLONE_UNTRACED	Do not let the tracing process force CLONE_PTRACE on the child.
CLONE_STOP	Start process in the TASK_STOPPED state.
CLONE_SETTLS	Create a new TLS (thread-local storage) for the child.
CLONE_CHILD_CLEARTID	Clear the TID in the child.
CLONE_CHILD_SETTID	Set the TID in the child.
CLONE_PARENT_SETTID	Set the TID in the parent.
CLONE_VM	Parent and child share address space.

Kernel Threads

It is often useful for the kernel to perform some operations in the background. The kernel accomplishes this via **kernel threads**—standard processes that exist solely in kernel-space. The significant difference between kernel threads and normal processes is that kernel threads do not have an address space. (Their `mm` pointer, which points at their address space, is `NULL`.) They operate only in kernel-space and do not context switch into user-space. Kernel threads, however, are schedulable and preemptable, the same as normal processes.

Linux delegates several tasks to kernel threads, most notably the **flush** tasks and the **ksoftirqd** task. You can see the kernel threads on your Linux system by running the command `ps -ef`. There are a lot of them! Kernel threads are created on system boot by other kernel threads. Indeed, a kernel thread can be created only by another kernel thread. The kernel handles this automatically by forking all new kernel threads off of the

kthreadd kernel process. The interface, declared in `<linux/kthread.h>`, for spawning a new kernel thread from an existing one is

```
struct task_struct *kthread_create(int (*threadfn)(void *data),
                                   void *data,
                                   const char namefmt[],
                                   ...)
```

The new task is created via the `clone()` system call by the *kthread* kernel process. The new process will run the `threadfn` function, which is passed the `data` argument. The process will be named `namefmt`, which takes *printf*-style formatting arguments in the variable argument list. The process is created in an unrunnable state; it will not start running until explicitly woken up via `wake_up_process()`. A process can be created and made runnable with a single function, `kthread_run()`:

```
struct task_struct *kthread_run(int (*threadfn)(void *data),
                                void *data,
                                const char namefmt[],
                                ...)
```

This routine, implemented as a macro, simply calls both `kthread_create()` and `wake_up_process()`:

```
#define kthread_run(threadfn, data, namefmt, ...) \
({ \
    struct task_struct *k; \
 \
    k = kthread_create(threadfn, data, namefmt, ## __VA_ARGS__); \
    if (!IS_ERR(k)) \
        wake_up_process(k); \
 \
    k; \
})
```

When started, a kernel thread continues to exist until it calls `do_exit()` or another part of the kernel calls `kthread_stop()`, passing in the address of the `task_struct` structure returned by `kthread_create()`:

```
int kthread_stop(struct task_struct *k)
```

We discuss specific kernel threads in more detail in later chapters.

Process Termination

It is sad, but eventually processes must die. When a process terminates, the kernel releases the resources owned by the process and notifies the child's parent of its demise.

Generally, process destruction is self-induced. It occurs when the process calls the `exit()` system call, either explicitly when it is ready to terminate or implicitly on return from the main subroutine of any program. (That is, the C compiler places a call to `exit()` after `main()` returns.) A process can also terminate involuntarily. This occurs when the

process receives a signal or exception it cannot handle or ignore. Regardless of how a process terminates, the bulk of the work is handled by `do_exit()`, defined in `kernel/exit.c`, which completes a number of chores:

1. It sets the `PF_EXITING` flag in the `flags` member of the `task_struct`.
2. It calls `del_timer_sync()` to remove any kernel timers. Upon return, it is guaranteed that no timer is queued and that no timer handler is running.
3. If BSD process accounting is enabled, `do_exit()` calls `acct_update_integrals()` to write out accounting information.
4. It calls `exit_mm()` to release the `mm_struct` held by this process. If no other process is using this address space—that is, if the address space is not shared—the kernel then destroys it.
5. It calls `exit_sem()`. If the process is queued waiting for an IPC semaphore, it is dequeued here.
6. It then calls `exit_files()` and `exit_fs()` to decrement the usage count of objects related to file descriptors and filesystem data, respectively. If either usage counts reach zero, the object is no longer in use by any process, and it is destroyed.
7. It sets the task's exit code, stored in the `exit_code` member of the `task_struct`, to the code provided by `exit()` or whatever kernel mechanism forced the termination. The exit code is stored here for optional retrieval by the parent.
8. It calls `exit_notify()` to send signals to the task's parent, reparents any of the task's children to another thread in their thread group or the init process, and sets the task's exit state, stored in `exit_state` in the `task_struct` structure, to `EXIT_ZOMBIE`.
9. `do_exit()` calls `schedule()` to switch to a new process (see Chapter 4). Because the process is now not schedulable, this is the last code the task will ever execute. `do_exit()` never returns.

At this point, all objects associated with the task (assuming the task was the sole user) are freed. The task is not runnable (and no longer has an address space in which to run) and is in the `EXIT_ZOMBIE` exit state. The only memory it occupies is its kernel stack, the `thread_info` structure, and the `task_struct` structure. The task exists solely to provide information to its parent. After the parent retrieves the information, or notifies the kernel that it is uninterested, the remaining memory held by the process is freed and returned to the system for use.

Removing the Process Descriptor

After `do_exit()` completes, the process descriptor for the terminated process still exists, but the process is a zombie and is unable to run. As discussed, this enables the system to obtain information about a child process after it has terminated. Consequently, the acts of

cleaning up after a process and removing its process descriptor are separate. After the parent has obtained information on its terminated child, or signified to the kernel that it does not care, the child's `task_struct` is deallocated.

The `wait()` family of functions are implemented via a single (and complicated) system call, `wait4()`. The standard behavior is to suspend execution of the calling task until one of its children exits, at which time the function returns with the PID of the exited child. Additionally, a pointer is provided to the function that on return holds the exit code of the terminated child.

When it is time to finally deallocate the process descriptor, `release_task()` is invoked. It does the following:

1. It calls `__exit_signal()`, which calls `__unhash_process()`, which in turns calls `detach_pid()` to remove the process from the `pidhash` and remove the process from the task list.
2. `__exit_signal()` releases any remaining resources used by the now dead process and finalizes statistics and bookkeeping.
3. If the task was the last member of a thread group, and the leader is a zombie, then `release_task()` notifies the zombie leader's parent.
4. `release_task()` calls `put_task_struct()` to free the pages containing the process's kernel stack and `thread_info` structure and deallocate the slab cache containing the `task_struct`.

At this point, the process descriptor and all resources belonging solely to the process have been freed.

The Dilemma of the Parentless Task

If a parent exits before its children, some mechanism must exist to **reparent** any child tasks to a new process, or else parentless terminated processes would forever remain zombies, wasting system memory. The solution is to reparent a task's children on exit to either another process in the current thread group or, if that fails, the `init` process. `do_exit()` calls `exit_notify()`, which calls `forget_original_parent()`, which, in turn, calls `find_new_reaper()` to perform the reparenting:

```
static struct task_struct *find_new_reaper(struct task_struct *father)
{
    struct pid_namespace *pid_ns = task_active_pid_ns(father);
    struct task_struct *thread;

    thread = father;
    while_each_thread(father, thread) {
        if (thread->flags & PF_EXITING)
            continue;
        if (unlikely(pid_ns->child_reaper == father))
```

```

        pid_ns->child_reaper = thread;
    return thread;
}

if (unlikely(pid_ns->child_reaper == father)) {
    write_unlock_irq(&tasklist_lock);
    if (unlikely(pid_ns == &init_pid_ns))
        panic("Attempted to kill init!");

    zap_pid_ns_processes(pid_ns);
    write_lock_irq(&tasklist_lock);
    /*
     * We can not clear ->child_reaper or leave it alone.
     * There may be stealth EXIT_DEAD tasks on ->children,
     * forget_original_parent() must move them somewhere.
     */
    pid_ns->child_reaper = init_pid_ns.child_reaper;
}
return pid_ns->child_reaper;
}

```

This code attempts to find and return another task in the process's thread group. If another task is not in the thread group, it finds and returns the `init` process. Now that a suitable new parent for the children is found, each child needs to be located and re-parented to reaper:

```

reaper = find_new_reaper(father);
list_for_each_entry_safe(p, n, &father->children, sibling) {
    p->real_parent = reaper;
    if (p->parent == father) {
        BUG_ON(p->ptrace);
        p->parent = p->real_parent;
    }
    reparent_thread(p, father);
}

```

`ptrace_exit_finish()` is then called to do the same reparenting but to a list of *ptraced* children:

```

void exit_ptrace(struct task_struct *tracer)
{
    struct task_struct *p, *n;
    LIST_HEAD(ptrace_dead);

    write_lock_irq(&tasklist_lock);
    list_for_each_entry_safe(p, n, &tracer->ptraced, ptrace_entry) {
        if (!__ptrace_detach(tracer, p))
            list_add(&p->ptrace_entry, &ptrace_dead);
    }
}

```

```

    }
    write_unlock_irq(&tasklist_lock);

    BUG_ON(!list_empty(&tracer->ptraced));

    list_for_each_entry_safe(p, n, &ptrace_dead, ptrace_entry) {
        list_del_init(&p->ptrace_entry);
        release_task(p);
    }
}

```

The rationale behind having both a child list and a ptraced list is interesting; it is a new feature in the 2.6 kernel. When a task is *ptraced*, it is temporarily reparented to the debugging process. When the task's parent exits, however, it must be reparented along with its other siblings. In previous kernels, this resulted in a loop over *every process in the system* looking for children. The solution is simply to keep a separate list of a process's children being ptraced—reducing the search for one's children from every process to just two relatively small lists.

With the process successfully reparented, there is no risk of stray zombie processes. The *init* process routinely calls `wait()` on its children, cleaning up any zombies assigned to it.

Conclusion

In this chapter, we looked at the core operating system abstraction of the *process*. We discussed the generalities of the process, why it is important, and the relationship between processes and threads. We then discussed how Linux stores and represents processes (with `task_struct` and `thread_info`), how processes are created (via `fork()` and ultimately `clone()`), how new executable images are loaded into address spaces (via the `exec()` family of system calls), the hierarchy of processes, how parents glean information about their deceased children (via the `wait()` family of system calls), and how processes ultimately die (forcefully or intentionally via `exit()`). The process is a fundamental and crucial abstraction, at the heart of every modern operating system, and ultimately the reason we have operating systems altogether (to run programs).

The next chapter discusses process scheduling, which is the delicate and interesting manner in which the kernel decides which processes to run, at what time, and in what order.

Process Scheduling

The prior chapter discussed *processes*, the operating system abstraction of active program code. This chapter discusses the *process scheduler*, the *kernel subsystem* that puts those processes to work.

The process scheduler decides which process runs, when, and for how long. The process scheduler (or simply the *scheduler*, to which it is often shortened) divides the finite resource of processor time between the runnable processes on a system. The scheduler is the basis of a multitasking operating system such as Linux. By deciding which process runs next, the scheduler is responsible for best utilizing the system and giving users the impression that multiple processes are executing simultaneously.

The idea behind the scheduler is simple. To best utilize processor time, assuming there are *runnable* processes, a process should always be running. If there are more runnable processes than processors in a system, some processes will not be running at a given moment. These processes are *waiting to run*. Deciding which process runs next, given a set of runnable processes, is the fundamental decision that the scheduler must make.

Multitasking

A *multitasking* operating system is one that can simultaneously interleave execution of more than one process. On single processor machines, this gives the illusion of multiple processes running concurrently. On multiprocessor machines, such functionality enables processes to actually run concurrently, in parallel, on different processors. On either type of machine, it also enables many processes to *block* or *sleep*, not actually executing until work is available. These processes, although in memory, are not *runnable*. Instead, such processes utilize the kernel to wait until some event (keyboard input, network data, passage of time, and so on) occurs. Consequently, a modern Linux system can have many processes in memory but, say, only one in a runnable state.

Multitasking operating systems come in two flavors: cooperative multitasking and preemptive multitasking. Linux, like all Unix variants and most modern operating systems, implements preemptive multitasking. In preemptive multitasking, the scheduler decides when a process is to cease running and a new process is to begin running. The act of

involuntarily suspending a running process is called *preemption*. The time a process runs before it is preempted is usually predetermined, and it is called the *timeslice* of the process. The timeslice, in effect, gives each runnable process a *slice* of the processor's time. Managing the timeslice enables the scheduler to make global scheduling decisions for the system. It also prevents any one process from monopolizing the processor. On many modern operating systems, the timeslice is dynamically calculated as a function of process behavior and configurable system policy. As we shall see, Linux's unique "fair" scheduler does not employ timeslices *per se*, to interesting effect.

Conversely, in *cooperative multitasking*, a process does not stop running until it voluntarily decides to do so. The act of a process voluntarily suspending itself is called *yielding*. Ideally, processes yield often, giving each runnable process a decent chunk of the processor, but the operating system cannot enforce this. The shortcomings of this approach are manifest: The scheduler cannot make global decisions regarding how long processes run; processes can monopolize the processor for longer than the user desires; and a hung process that never yields can potentially bring down the entire system. Thankfully, most operating systems designed in the last two decades employ preemptive multitasking, with Mac OS 9 (and earlier) and Windows 3.1 (and earlier) being the most notable (and embarrassing) exceptions. Of course, Unix has sported preemptive multitasking since its inception.

Linux's Process Scheduler

From Linux's first version in 1991 through the 2.4 kernel series, the Linux scheduler was simple, almost pedestrian, in design. It was easy to understand, but scaled poorly in light of many runnable processes or many processors.

In response, during the 2.5 kernel development series, the Linux kernel received a scheduler overhaul. A new scheduler, commonly called the *$O(1)$ scheduler* because of its algorithmic behavior,¹ solved the shortcomings of the previous Linux scheduler and introduced powerful new features and performance characteristics. By introducing a constant-time algorithm for timeslice calculation and per-processor runqueues, it rectified the design limitations of the earlier scheduler.

The $O(1)$ scheduler performed admirably and scaled effortlessly as Linux supported large "iron" with tens if not hundreds of processors. Over time, however, it became evident that the $O(1)$ scheduler had several pathological failures related to scheduling latency-sensitive applications. These applications, called *interactive processes*, include any application with which the user interacts. Thus, although the $O(1)$ scheduler was ideal for large server workloads—which lack interactive processes—it performed below par on desktop systems, where interactive applications are the *raison d'être*. Beginning early in the

¹ $O(1)$ is an example of big-o notation. In short, it means the scheduler can perform its work in constant time, regardless of the size of any inputs. A full explanation of big-o notation is in Chapter 6, "Kernel Data Structures."

2.6 kernel series, developers introduced new process schedulers aimed at improving the interactive performance of the $O(1)$ scheduler. The most notable of these was the **Rotating Staircase Deadline** scheduler, which introduced the concept of *fair scheduling*, borrowed from queuing theory, to Linux's process scheduler. This concept was the inspiration for the $O(1)$ scheduler's eventual replacement in kernel version 2.6.23, the **Completely Fair Scheduler**, or **CFS**.

This chapter discusses the fundamentals of scheduler design and how they apply to the Completely Fair Scheduler and its goals, design, implementation, algorithms, and related system calls. We also discuss the $O(1)$ scheduler because its implementation is a more "classic" Unix process scheduler model.

Policy

Policy is the behavior of the scheduler that determines what runs when. A scheduler's policy often determines the overall feel of a system and is responsible for optimally utilizing processor time. Therefore, it is very important.

I/O-Bound Versus Processor-Bound Processes

Processes can be classified as either **I/O-bound** or **processor-bound**. The former is characterized as a process that spends much of its time submitting and waiting on I/O requests. Consequently, such a process is runnable for only short durations, because it eventually blocks waiting on more I/O. (Here, by I/O, we mean any type of blockable resource, such as keyboard input or network I/O, and not just disk I/O.) Most graphical user interface (GUI) applications, for example, are I/O-bound, even if they never read from or write to the disk, because they spend most of their time waiting on user interaction via the keyboard and mouse.

Conversely, processor-bound processes spend much of their time executing code. They tend to run until they are preempted because they do not block on I/O requests very often. Because they are not I/O-driven, however, system response does not dictate that the scheduler run them often. A scheduler policy for processor-bound processes, therefore, tends to run such processes less frequently but for longer durations. The ultimate example of a processor-bound process is one executing an infinite loop. More palatable examples include programs that perform a lot of mathematical calculations, such as ssh-keygen or MATLAB.

Of course, these classifications are not mutually exclusive. Processes can exhibit both behaviors simultaneously: The X Window server, for example, is both processor and I/O-intense. Other processes can be I/O-bound but dive into periods of intense processor action. A good example of this is a word processor, which normally sits waiting for key presses but at any moment might peg the processor in a rabid fit of spell checking or macro calculation.

The scheduling policy in a system must attempt to satisfy two conflicting goals: fast process response time (low latency) and maximal system utilization (high throughput). To

satisfy these at-odds requirements, schedulers often employ complex algorithms to determine the most worthwhile process to run while not compromising fairness to other, lower priority, processes. The scheduler policy in Unix systems tends to explicitly favor I/O-bound processes, thus providing good process response time. Linux, aiming to provide good interactive response and desktop performance, optimizes for process response (low latency), thus favoring I/O-bound processes over processor-bound processors. As we will see, this is done in a creative manner that does not neglect processor-bound processes.

Process Priority

A common type of scheduling algorithm is priority-based scheduling. The goal is to rank processes based on their worth and need for processor time. The general idea, which isn't exactly implemented on Linux, is that processes with a higher priority run before those with a lower priority, whereas processes with the same priority are scheduled round-robin (one after the next, repeating). On some systems, processes with a higher priority also receive a longer timeslice. The runnable process with timeslice remaining and the highest priority always runs. Both the user and the system can set a process's priority to influence the scheduling behavior of the system.

The Linux kernel implements two separate priority ranges. The first is the nice value, a number from -20 to +19 with a default of 0. Larger nice values correspond to a lower priority—you are being “nice” to the other processes on the system. Processes with a lower nice value (higher priority) receive a larger proportion of the system's processor compared to processes with a higher nice value (lower priority). Nice values are the standard priority range used in all Unix systems, although different Unix systems apply them in different ways, reflective of their individual scheduling algorithms. In other Unix-based systems, such as Mac OS X, the nice value is a control over the *absolute* timeslice allotted to a process; in Linux, it is a control over the *proportion* of timeslice. You can see a list of the processes on your system and their respective nice values (under the column marked *NI*) with the command `ps -el.`

The second range is the real-time priority. The values are configurable, but by default range from 0 to 99, inclusive. Opposite from nice values, higher real-time priority values correspond to a greater priority. All real-time processes are at a higher priority than normal processes; that is, the real-time priority and nice value are in disjoint value spaces. Linux implements real-time priorities in accordance with the relevant Unix standards, specifically POSIX.1b. All modern Unix systems implement a similar scheme. You can see a list of the processes on your system and their respective real-time priority (under the column marked *RTPRIO*) with the command

```
ps -eo state,uid,pid,ppid,rtprio,time,comm.
```

A value of “-” means the process is not real-time.

Timeslice

The timeslice² is the numeric value that represents how long a task can run until it is preempted. The scheduler policy must dictate a default timeslice, which is not a trivial exercise. Too long a timeslice causes the system to have poor interactive performance; the system will no longer feel as if applications are concurrently executed. Too short a timeslice causes significant amounts of processor time to be wasted on the overhead of switching processes because a significant percentage of the system's time is spent switching from one process with a short timeslice to the next. Furthermore, the conflicting goals of I/O-bound versus processor-bound processes again arise: I/O-bound processes do not need longer timeslices (although they do like to run often), whereas processor-bound processes crave long timeslices (to keep their caches hot).

With this argument, it would seem that *any* long timeslice would result in poor interactive performance. In many operating systems, this observation is taken to heart, and the default timeslice is rather low—for example, 10 milliseconds. Linux's CFS scheduler, however, does not directly assign timeslices to processes. Instead, in a novel approach, CFS assigns processes a proportion of the processor. On Linux, therefore, the amount of processor time that a process receives is a function of the load of the system. This assigned proportion is further affected by each process's nice value. The nice value acts as a weight, changing the proportion of the processor time each process receives. Processes with higher nice values (a lower priority) receive a deflationary weight, yielding them a smaller proportion of the processor; processes with smaller nice values (a higher priority) receive an inflationary weight, netting them a larger proportion of the processor.

As mentioned, the Linux operating system is *preemptive*. When a process enters the runnable state, it becomes eligible to run. In most operating systems, whether the process runs immediately, preempting the currently running process, is a function of the process's priority and available timeslice. In Linux, under the new CFS scheduler, the decision is a function of how much of a proportion of the processor the newly runnable processor has consumed. If it has consumed a smaller proportion of the processor than the currently executing process, it runs immediately, preempting the current process. If not, it is scheduled to run at a later time.

The Scheduling Policy in Action

Consider a system with two runnable tasks: a text editor and a video encoder. The text editor is I/O-bound because it spends nearly all its time waiting for user key presses. (No matter how fast the user types, it is not *that* fast.) Despite this, when the text editor does receive a key press, the user expects the editor to respond immediately. Conversely, the video encoder is processor-bound. Aside from reading the raw data stream from the disk

² Timeslice is sometimes called quantum or processor slice in other systems. Linux calls it timeslice, thus so should you.

and later writing the resulting video, the encoder spends all its time applying the video codec to the raw data, easily consuming 100% of the processor. The video encoder does not have any strong time constraints on when it runs—if it started running now or in half a second, the user could not tell and would not care. Of course, the sooner it finishes the better, but latency is not a primary concern.

In this scenario, ideally the scheduler gives the text editor a larger proportion of the available processor than the video encoder, because the text editor is interactive. We have two goals for the text editor. First, we want it to have a large amount of processor time available to it; not because it needs a lot of processor (it does not) but because we want it to always have processor time available the moment it needs it. Second, we want the text editor to preempt the video encoder the moment it wakes up (say, when the user presses a key). This can ensure the text editor has good *interactive performance* and is responsive to user input. On most operating systems, these goals are accomplished (if at all) by giving the text editor a higher priority and larger timeslice than the video encoder. Advanced operating systems do this automatically, by detecting that the text editor is interactive. Linux achieves these goals too, but by different means. Instead of assigning the text editor a specific priority and timeslice, it guarantees the text editor a specific proportion of the processor. If the video encoder and text editor are the only running processes and both are at the same nice level, this proportion would be 50%—each would be guaranteed half of the processor's time. Because the text editor spends most of its time blocked, waiting for user key presses, it does not use anywhere near 50% of the processor. Conversely, the video encoder is free to use *more* than its allotted 50%, enabling it to finish the encoding quickly.

The crucial concept is what happens when the text editor wakes up. Our primary goal is to ensure it runs immediately upon user input. In this case, when the editor wakes up, CFS notes that it is allotted 50% of the processor but has used considerably less. Specifically, CFS determines that the text editor has run for *less time* than the video encoder. Attempting to give all processes a *fair share* of the processor, it then preempts the video encoder and enables the text editor to run. The text editor runs, quickly processes the user's key press, and again sleeps, waiting for more input. As the text editor has not consumed its allotted 50%, we continue in this manner, with CFS always enabling the text editor to run when it wants and the video encoder to run the rest of the time.

The Linux Scheduling Algorithm

In the previous sections, we discussed process scheduling in the abstract, with only occasional mention of how Linux applies a given concept to reality. With the foundation of scheduling now built, we can dive into Linux's process scheduler.

Scheduler Classes

The Linux scheduler is modular, enabling different algorithms to schedule different types of processes. This modularity is called scheduler classes. Scheduler classes enable different, pluggable algorithms to coexist, scheduling their own types of processes. Each scheduler

class has a priority. The base scheduler code, which is defined in `kernel/sched.c`, iterates over each scheduler class in order of priority. The highest priority scheduler class that has a runnable process wins, selecting who runs next.

The Completely Fair Scheduler (CFS) is the registered scheduler class for normal processes, called `SCHED_NORMAL` in Linux (and `SCHED_OTHER` in POSIX). CFS is defined in `kernel/sched_fair.c`. The rest of this section discusses the CFS algorithm and is germane to any Linux kernel since 2.6.23. We discuss the scheduler class for real-time processes in a later section.

Process Scheduling in Unix Systems

To discuss fair scheduling, we must first describe how traditional Unix systems schedule processes. As mentioned in the previous section, modern process schedulers have two common concepts: process priority and timeslice. Timeslice is how long a process runs; processes start with some default timeslice. Processes with a higher priority run more frequently and (on many systems) receive a higher timeslice. On Unix, the priority is exported to user-space in the form of nice values. This sounds simple, but in practice it leads to several pathological problems, which we now discuss.

First, mapping nice values onto timeslices requires a decision about what absolute timeslice to allot each nice value. This leads to suboptimal switching behavior. For example, let's assume we assign processes of the default nice value (zero) a timeslice of 100 milliseconds and processes at the highest nice value (+20, the lowest priority) a timeslice of 5 milliseconds. Further, let's assume one of each of these processes is runnable. Our default-priority process thus receives 20/21 (100 out of 105 milliseconds) of the processor, whereas our low priority process receives 1/21 (5 out of 105 milliseconds) of the processor. We could have used any numbers for this example, but we assume this allotment is optimal since we chose it. Now, what happens if we run exactly two low priority processes? We'd expect they each receive 50% of the processor, which they do. But they each enjoy the processor for only 5 milliseconds at a time (5 out of 10 milliseconds each)! That is, instead of context switching twice every 105 milliseconds, we now context switch twice every 10 milliseconds. Conversely, if we have two normal priority processes, each again receives the correct 50% of the processor, but in 100 millisecond increments. Neither of these timeslice allotments are necessarily ideal; each is simply a byproduct of a given nice value to timeslice mapping coupled with a specific runnable process priority mix. Indeed, given that high nice value (low priority) processes tend to be background, processor-intensive tasks, while normal priority processes tend to be foreground user tasks, this timeslice allotment is exactly *backward* from ideal!

A second problem concerns relative nice values and again the nice value to timeslice mapping. Say we have two processes, each a single nice value apart. First, let's assume they are at nice values 0 and 1. This might map (and indeed did in the $O(1)$ scheduler) to timeslices of 100 and 95 milliseconds, respectively. These two values are nearly identical, and thus the difference here between a single nice value is small. Now, instead, let's assume our two processes are at nice values of 18 and 19. This now maps to timeslices of

10 and 5 milliseconds, respectively—the former receiving twice the processor time as the latter! Because nice values are most commonly used in relative terms (as the system call accepts an increment, not an absolute value), this behavior means that “nicing down a process by one” has wildly different effects depending on the starting nice value.

Third, if performing a nice value to timeslice mapping, we need the ability to assign an absolute timeslice. This absolute value must be measured in terms the kernel can measure. In most operating systems, this means the timeslice must be some integer multiple of the timer tick. (See Chapter 11, “Timers and Time Management,” for a discussion on time.) This introduces several problems. First, the minimum timeslice has a floor of the period of the timer tick, which might be as high as 10 milliseconds or as low as 1 millisecond. Second, the system timer limits the difference between two timeslices; successive nice values might map to timeslices as much as 10 milliseconds or as little as 1 millisecond apart. Finally, timeslices change with different timer ticks. (If this paragraph’s discussion of timer ticks is foreign, reread it after reading Chapter 11. This is only one motivation behind CFS.)

The fourth and final problem concerns handling process wake up in a priority-based scheduler that wants to optimize for interactive tasks. In such a system, you might want to give freshly woken-up tasks a priority boost by allowing them to run immediately, even if their timeslice was expired. Although this improves interactive performance in many, if not most, situations, it also opens the door to pathological cases where certain sleep/wake up use cases can game the scheduler into providing one process an unfair amount of processor time, at the expense of the rest of the system.

Most of these problems are solvable by making substantial but not paradigm-shifting changes to the old-school Unix scheduler. For example, making nice values geometric instead of additive solves the second problem. And mapping nice values to timeslices using a measurement decoupled from the timer tick solves the third problem. But such solutions belie the true problem, which is that assigning absolute timeslices yields a constant switching rate but variable fairness. The approach taken by CFS is a radical (for process schedulers) rethinking of timeslice allotment: Do away with timeslices completely and assign each process a proportion of the processor. CFS thus yields constant fairness but a variable switching rate.

Fair Scheduling

CFS is based on a simple concept: Model process scheduling as if the system had an ideal, perfectly multitasking processor. In such a system, each process would receive $1/n$ of the processor’s time, where n is the number of runnable processes, and we’d schedule them for infinitely small durations, so that in any measurable period we’d have run all n processes for the same amount of time. As an example, assume we have two processes. In the standard Unix model, we might run one process for 5 milliseconds and then another process for 5 milliseconds. While running, each process would receive 100% of the processor. In an ideal, perfectly multitasking processor, we would run both processes *simultaneously* for 10 milliseconds, each at 50% power. This latter model is called *perfect multitasking*.

Of course, such a model is also impractical, because it is not possible on a single processor to *literally* run multiple processes simultaneously. Moreover, it is not efficient to run processes for infinitely small durations. That is, there is a *switching cost* to preempting one process for another: the overhead of swapping one process for another and the effects on caches, for example. Thus, although we'd like to run processes for very small durations, CFS is mindful of the overhead and performance hit in doing so. Instead, CFS will run each process for some amount of time, round-robin, selecting next the process that has run the least. Rather than assign each process a timeslice, CFS calculates how long a process should run as a function of the total number of runnable processes. Instead of using the nice value to calculate a timeslice, CFS uses the nice value to *weight* the proportion of processor a process is to receive: Higher valued (lower priority) processes receive a fractional weight relative to the default nice value, whereas lower valued (higher priority) processes receive a larger weight.

Each process then runs for a “timeslice” proportional to its weight divided by the total weight of all runnable threads. To calculate the actual timeslice, CFS sets a target for its approximation of the “infinitely small” scheduling duration in perfect multitasking. This target is called the *targeted latency*. Smaller targets yield better interactivity and a closer approximation to perfect multitasking, at the expense of higher switching costs and thus worse overall throughput. Let's assume the targeted latency is 20 milliseconds and we have two runnable tasks at the same priority. *Regardless of those task's priority*, each will run for 10 milliseconds before preempting in favor of the other. If we have four tasks at the same priority, each will run for 5 milliseconds. If there are 20 tasks, each will run for 1 millisecond.

Note as the number of runnable tasks approaches infinity, the proportion of allotted processor and the assigned timeslice approaches zero. As this will eventually result in unacceptable switching costs, CFS imposes a floor on the timeslice assigned to each process. This floor is called the *minimum granularity*. By default it is 1 millisecond. Thus, even as the number of runnable processes approaches infinity, each will run for at least 1 millisecond, to ensure there is a ceiling on the incurred switching costs. (Astute readers will note that CFS is thus not perfectly fair when the number of processes grows so large that the calculated proportion is floored by the minimum granularity. This is true. Although modifications to fair queuing exist to improve upon this fairness, CFS was explicitly designed to make this trade-off. In the common case of only a handful of runnable processes, CFS is perfectly fair.)

Now, let's again consider the case of two runnable processes, except with dissimilar nice values—say, one with the default nice value (zero) and one with a nice value of 5. These nice values have dissimilar weights and thus our two processes receive different proportions of the processor's time. In this case, the weights work out to about a 1/3 penalty for the nice-5 process. If our target latency is again 20 milliseconds, our two processes will receive 15 milliseconds and 5 milliseconds each of processor time, respectively. Say our two runnable processes instead had nice values of 10 and 15. What would be the allotted timeslices? Again 15 and 5 milliseconds each! Absolute nice values no

longer affect scheduling decisions; only relative values affect the proportion of processor time allotted.

Put generally, the proportion of processor time that any process receives is determined only by the relative difference in niceness between it and the other runnable processes. The nice values, instead of yielding additive increases to timeslices, yield geometric differences. The absolute timeslice allotted any nice value is not an absolute number, but a given proportion of the processor. CFS is called a *fair scheduler* because it gives each process a fair share—a proportion—of the processor's time. As mentioned, note that CFS isn't perfectly fair, because it only approximates perfect multitasking, but it *can* place a lower bound on latency of n for n runnable processes on the unfairness.

The Linux Scheduling Implementation

With the discussion of the motivation for and logic of CFS behind us, we can now explore CFS's actual implementation, which lives in `kernel/sched_fair.c`. Specifically, we discuss four components of CFS:

- Time Accounting
- Process Selection
- The Scheduler Entry Point
- Sleeping and Waking Up

Time Accounting

All process schedulers must account for the time that a process runs. Most Unix systems do so, as discussed earlier, by assigning each process a timeslice. On each tick of the system clock, the timeslice is decremented by the tick period. When the timeslice reaches zero, the process is preempted in favor of another runnable process with a nonzero timeslice.

The Scheduler Entity Structure

CFS does not have the notion of a timeslice, but it must still keep account for the time that each process runs, because it needs to ensure that each process runs only for its fair share of the processor. CFS uses the *scheduler entity structure*, `struct sched_entity`, defined in `<linux/sched.h>`, to keep track of process accounting:

```
struct sched_entity {
    struct load_weight    load;
    struct rb_node        run_node;
    struct list_head     group_node;
    unsigned int         on_rq;
    u64                  exec_start;
    u64                  sum_exec_runtime;
    u64                  vruntime;
    u64                  prev_sum_exec_runtime;
```



```

u64                last_wakeup;
u64                avg_overlap;
u64                nr_migrations;
u64                start_runtime;
u64                avg_wakeup;

/* many stat variables elided, enabled only if CONFIG_SCHEDSTATS is set */
};

```

The scheduler entity structure is embedded in the *process descriptor*, `struct task_struct`, as a member variable named `se`. We discussed the process descriptor in Chapter 3, “Process Management.”

The Virtual Runtime

The `vruntime` variable stores the *virtual runtime* of a process, which is the actual runtime (the amount of time spent running) normalized (or weighted) by the number of runnable processes. The virtual runtime’s units are nanoseconds and therefore `vruntime` is decoupled from the timer tick. The virtual runtime is used to help us approximate the “ideal multitasking processor” that CFS is modeling. With such an ideal processor, we wouldn’t need `vruntime`, because all runnable processes would perfectly multitask. That is, on an ideal processor, the virtual runtime of all processes of the same priority would be identical—all tasks would have received an equal, fair share of the processor. Because processors are not capable of perfect multitasking and we must run each process in succession, CFS uses `vruntime` to account for how long a process has run and thus how much longer it ought to run.

The function `update_curr()`, defined in `kernel/sched_fair.c`, manages this accounting:

```

static void update_curr(struct cfs_rq *cfs_rq)
{
    struct sched_entity *curr = cfs_rq->curr;
    u64 now = rq_of(cfs_rq)->clock;
    unsigned long delta_exec;

    if (unlikely(!curr))
        return;

    /*
     * Get the amount of time the current task was running
     * since the last time we changed load (this cannot
     * overflow on 32 bits):
     */
    delta_exec = (unsigned long)(now - curr->exec_start);
    if (!delta_exec)
        return;
}

```

```

__update_curr(cfs_rq, curr, delta_exec);
curr->exec_start = now;

if (entity_is_task(curr)) {
    struct task_struct *curtask = task_of(curr);

    trace_sched_stat_runtime(curtask, delta_exec, curr->vruntime);
    cpuacct_charge(curtask, delta_exec);
    account_group_exec_runtime(curtask, delta_exec);
}
}

```

`update_curr()` calculates the execution time of the current process and stores that value in `delta_exec`. It then passes that runtime to `__update_curr()`, which weights the time by the number of runnable processes. The current process's `vruntime` is then incremented by the weighted value:

```

/*
 * Update the current task's runtime statistics. Skip current tasks that
 * are not in our scheduling class.
 */
static inline void
__update_curr(struct cfs_rq *cfs_rq, struct sched_entity *curr,
              unsigned long delta_exec)
{
    unsigned long delta_exec_weighted;

    schedstat_set(curr->exec_max, max((u64)delta_exec, curr->exec_max));

    curr->sum_exec_runtime += delta_exec;
    schedstat_add(cfs_rq, exec_clock, delta_exec);
    delta_exec_weighted = calc_delta_fair(delta_exec, curr);

    curr->vruntime += delta_exec_weighted;
    update_min_vruntime(cfs_rq);
}

```

`update_curr()` is invoked periodically by the system timer and also whenever a process becomes runnable or blocks, becoming unrunnable. In this manner, `vruntime` is an accurate measure of the runtime of a given process and an indicator of what process should run next.

Process Selection

In the last section, we discussed how `vruntime` on an ideal, perfectly multitasking processor would be identical among all runnable processes. In reality, we cannot perfectly multitask, so CFS attempts to balance a process's virtual runtime with a simple rule: When CFS

is deciding what process to run next, it picks the process with the smallest `vruntime`. This is, in fact, the core of CFS's scheduling algorithm: Pick the task with the smallest `vruntime`. That's it! The rest of this subsection describes how the selection of the process with the smallest `vruntime` is implemented.

CFS uses a *red-black tree* to manage the list of runnable processes and efficiently find the process with the smallest `vruntime`. A red-black tree, called an *rbtree* in Linux, is a type of *self-balancing binary search tree*. We discuss self-balancing binary search trees in general and red-black trees in particular in Chapter 6. For now, if you are unfamiliar, you need to know only that red-black trees are a data structure that store *nodes* of arbitrary data, identified by a specific *key*, and that they enable efficient search for a given *key*. (Specifically, obtaining a node identified by a given key is logarithmic in time as a function of total nodes in the tree.)

Picking the Next Task

Let's start with the assumption that we have a red-black tree populated with every runnable process in the system where the key for each node is the runnable process's virtual runtime. We'll look at how we build that tree in a moment, but for now let's assume we have it. Given this tree, the process that CFS wants to run next, which is the process with the smallest `vruntime`, is the leftmost node in the tree. That is, if you follow the tree from the root down through the left child, and continue moving to the left until you reach a leaf node, you find the process with the smallest `vruntime`. (Again, if you are unfamiliar with binary search trees, don't worry. Just know that this process is efficient.) CFS's process selection algorithm is thus summed up as "run the process represented by the leftmost node in the *rbtree*." The function that performs this selection is

```
__pick_next_entity(), defined in kernel/sched_fair.c:

static struct sched_entity *__pick_next_entity(struct cfs_rq *cfs_rq)
{
    struct rb_node *left = cfs_rq->rb_leftmost;

    if (!left)
        return NULL;

    return rb_entry(left, struct sched_entity, run_node);
}
```

Note that `__pick_next_entity()` does not actually traverse the tree to find the leftmost node, because the value is cached by `rb_leftmost`. Although it is efficient to walk the tree to find the leftmost node— $O(\text{height of tree})$, which is $O(\log N)$ for N nodes if the tree is balanced—it is even easier to cache the leftmost node. The return value from this function is the process that CFS next runs. If the function returns `NULL`, there is no leftmost node, and thus no nodes in the tree. In that case, there are no runnable processes, and CFS schedules the idle task.

Adding Processes to the Tree

Now let's look at how CFS adds processes to the rbtree and caches the leftmost node. This would occur when a process becomes runnable (wakes up) or is first created via `fork()`, as discussed in Chapter 3. Adding processes to the tree is performed by `enqueue_entity()`:

```
static void
enqueue_entity(struct cfs_rq *cfs_rq, struct sched_entity *se, int flags)
{
    /*
     * Update the normalized vruntime before updating min_vruntime
     * through callig update_curr().
     */
    if (!(flags & ENQUEUE_WAKEUP) || (flags & ENQUEUE_MIGRATE))
        se->vruntime += cfs_rq->min_vruntime;

    /*
     * Update run-time statistics of the 'current'.
     */
    update_curr(cfs_rq);
    account_entity_enqueue(cfs_rq, se);

    if (flags & ENQUEUE_WAKEUP) {
        place_entity(cfs_rq, se, 0);
        enqueue_sleeper(cfs_rq, se);
    }

    update_stats_enqueue(cfs_rq, se);
    check_spread(cfs_rq, se);
    if (se != cfs_rq->curr)
        __enqueue_entity(cfs_rq, se);
}
```

This function updates the runtime and other statistics and then invokes `__enqueue_entity()` to perform the actual heavy lifting of inserting the entry into the red-black tree:

```
/*
 * Enqueue an entity into the rb-tree:
 */
static void __enqueue_entity(struct cfs_rq *cfs_rq, struct sched_entity *se)
{
    struct rb_node **link = &cfs_rq->tasks_timeline.rb_node;
    struct rb_node *parent = NULL;
    struct sched_entity *entry;
    s64 key = entity_key(cfs_rq, se);
    int leftmost = 1;
```

```

/*
 * Find the right place in the rbtree:
 */
while (*link) {
    parent = *link;
    entry = rb_entry(parent, struct sched_entity, run_node);
    /*
     * We dont care about collisions. Nodes with
     * the same key stay together.
     */
    if (key < entity_key(cfs_rq, entry)) {
        link = &parent->rb_left;
    } else {
        link = &parent->rb_right;
        leftmost = 0;
    }
}

/*
 * Maintain a cache of leftmost tree entries (it is frequently
 * used):
 */
if (leftmost)
    cfs_rq->rb_leftmost = &se->run_node;

rb_link_node(&se->run_node, parent, link);
rb_insert_color(&se->run_node, &cfs_rq->tasks_timeline);
}

```

Let's look at this function. The body of the `while()` loop traverses the tree in search of a matching key, which is the inserted process's `vruntime`. Per the rules of the balanced tree, it moves to the left child if the key is smaller than the current node's key and to the right child if the key is larger. If it ever moves to the right, even once, it knows the inserted process cannot be the new leftmost node, and it sets `leftmost` to zero. If it moves only to the left, `leftmost` remains one, and we have a new leftmost node and can update the cache by setting `rb_leftmost` to the inserted process. The loop terminates when we compare ourselves to a node that has no child in the direction we move; `link` is then `NULL` and the loop terminates. When out of the loop, the function calls `rb_link_node()` on the parent node, making the inserted process the new child. The function `rb_insert_color()` updates the self-balancing properties of the tree; we discuss the coloring in Chapter 6.

Removing Processes from the Tree

Finally, let's look at how CFS removes processes from the red-black tree. This happens when a process blocks (becomes unrunnable) or terminates (ceases to exist):

```
static void
dequeue_entity(struct cfs_rq *cfs_rq, struct sched_entity *se, int sleep)
{
    /*
     * Update run-time statistics of the 'current'.
     */
    update_curr(cfs_rq);

    update_stats_dequeue(cfs_rq, se);
    clear_buddies(cfs_rq, se);

    if (se != cfs_rq->curr)
        __dequeue_entity(cfs_rq, se);
    account_entity_dequeue(cfs_rq, se);
    update_min_vruntime(cfs_rq);

    /*
     * Normalize the entity after updating the min_vruntime because the
     * update can refer to the ->curr item and we need to reflect this
     * movement in our normalized position.
     */
    if (!sleep)
        se->vruntime -= cfs_rq->min_vruntime;
}
```

As with adding a process to the red-black tree, the real work is performed by a helper function, `__dequeue_entity()`:

```
static void __dequeue_entity(struct cfs_rq *cfs_rq, struct sched_entity *se)
{
    if (cfs_rq->rb_leftmost == &se->run_node) {
        struct rb_node *next_node;

        next_node = rb_next(&se->run_node);
        cfs_rq->rb_leftmost = next_node;
    }

    rb_erase(&se->run_node, &cfs_rq->tasks_timeline);
}
```

Removing a process from the tree is much simpler because the rbtree implementation provides the `rb_erase()` function that performs all the work. The rest of this function updates the `rb_leftmost` cache. If the process-to-remove is the leftmost node, the func-

tion invokes `rb_next()` to find what would be the next node in an in-order traversal. This is what will be the leftmost node when the current leftmost node is removed.

The Scheduler Entry Point

The main entry point into the process schedule is the function `schedule()`, defined in `kernel/sched.c`. This is the function that the rest of the kernel uses to invoke the process scheduler, deciding which process to run and then running it. `schedule()` is generic with respect to scheduler classes. That is, it finds the highest priority scheduler class with a runnable process and asks it what to run next. Given that, it should be no surprise that `schedule()` is simple. The only important part of the function—which is otherwise too uninteresting to reproduce here—is its invocation of `pick_next_task()`, also defined in `kernel/sched.c`. The `pick_next_task()` function goes through each scheduler class, starting with the highest priority, and selects the highest priority process in the highest priority class:

```
/*
 * Pick up the highest-prio task:
 */
static inline struct task_struct *
pick_next_task(struct rq *rq)
{
    const struct sched_class *class;
    struct task_struct *p;

    /*
     * Optimization: we know that if all tasks are in
     * the fair class we can call that function directly:
     */
    if (likely(rq->nr_running == rq->cfs.nr_running)) {
        p = fair_sched_class.pick_next_task(rq);
        if (likely(p))
            return p;
    }

    class = sched_class_highest;
    for ( ; ; ) {
        p = class->pick_next_task(rq);
        if (p)
            return p;

        /*
         * Will never be NULL as the idle class always
         * returns a non-NULL p:
         */
        class = class->next;
    }
}
```

Note the optimization at the beginning of the function. Because CFS is the scheduler class for normal processes, and most systems run mostly normal processes, there is a small hack to quickly select the next CFS-provided process if the number of runnable processes is equal to the number of CFS runnable processes (which suggests that all runnable processes are provided by CFS).

The core of the function is the `for()` loop, which iterates over each class in priority order, starting with the highest priority class. Each class implements the `pick_next_task()` function, which returns a pointer to its next runnable process or, if there is not one, `NULL`. The first class to return a non-`NULL` value has selected the next runnable process. CFS's implementation of `pick_next_task()` calls `pick_next_entity()`, which in turn calls the `__pick_next_entity()` function that we discussed in the previous section.

Sleeping and Waking Up

Tasks that are sleeping (blocked) are in a special nonrunnable state. This is important because without this special state, the scheduler would select tasks that did not want to run or, worse, sleeping would have to be implemented as busy looping. A task sleeps for a number of reasons, but always while it is waiting for some event. The event can be a specified amount of time, more data from a file I/O, or another hardware event. A task can also involuntarily go to sleep when it tries to obtain a contended semaphore in the kernel (this is covered in Chapter 9, “An Introduction to Kernel Synchronization”). A common reason to sleep is file I/O—for example, the task issued a `read()` request on a file, which needs to be read in from disk. As another example, the task could be waiting for keyboard input. Whatever the case, the kernel behavior is the same: The task marks itself as sleeping, puts itself on a wait queue, removes itself from the red-black tree of runnable, and calls `schedule()` to select a new process to execute. Waking back up is the inverse: The task is set as runnable, removed from the wait queue, and added back to the red-black tree.

As discussed in the previous chapter, two states are associated with sleeping, `TASK_INTERRUPTIBLE` and `TASK_UNINTERRUPTIBLE`. They differ only in that tasks in the `TASK_UNINTERRUPTIBLE` state ignore signals, whereas tasks in the `TASK_INTERRUPTIBLE` state wake up prematurely and respond to a signal if one is issued. Both types of sleeping tasks sit on a wait queue, waiting for an event to occur, and are not runnable.

Wait Queues

Sleeping is handled via wait queues. A wait queue is a simple list of processes waiting for an event to occur. Wait queues are represented in the kernel by `wake_queue_head_t`. Wait queues are created statically via `DECLARE_WAITQUEUE()` or dynamically via `init_waitqueue_head()`. Processes put themselves on a wait queue and mark themselves not runnable. When the event associated with the wait queue occurs, the processes on the queue are awakened. It is important to implement sleeping and waking correctly, to avoid race conditions.

Some simple interfaces for sleeping used to be in wide use. These interfaces, however, have races: It is possible to go to sleep *after* the condition becomes true. In that case, the task might sleep indefinitely. Therefore, the recommended method for sleeping in the kernel is a bit more complicated:

```
/* 'q' is the wait queue we wish to sleep on */
DEFINE_WAIT(wait);

add_wait_queue(q, &wait);
while (!condition) {    /* condition is the event that we are waiting for */
    prepare_to_wait(&q, &wait, TASK_INTERRUPTIBLE);
    if (signal_pending(current))
        /* handle signal */
        schedule();
}
finish_wait(&q, &wait);
```

The task performs the following steps to add itself to a wait queue:

1. Creates a wait queue entry via the macro `DEFINE_WAIT()`.
2. Adds itself to a wait queue via `add_wait_queue()`. This wait queue awakens the process when the condition for which it is waiting occurs. Of course, there needs to be code elsewhere that calls `wake_up()` on the queue when the event actually does occur.
3. Calls `prepare_to_wait()` to change the process state to either `TASK_INTERRUPTIBLE` or `TASK_UNINTERRUPTIBLE`. This function also adds the task back to the wait queue if necessary, which is needed on subsequent iterations of the loop.
4. If the state is set to `TASK_INTERRUPTIBLE`, a signal wakes the process up. This is called a *spurious wake up* (a wake-up not caused by the occurrence of the event). So check and handle signals.
5. When the task awakens, it again checks whether the condition is true. If it is, it exits the loop. Otherwise, it again calls `schedule()` and repeats.
6. Now that the condition is true, the task sets itself to `TASK_RUNNING` and removes itself from the wait queue via `finish_wait()`.

If the condition occurs before the task goes to sleep, the loop terminates, and the task does not erroneously go to sleep. Note that kernel code often has to perform various other tasks in the body of the loop. For example, it might need to release locks before calling `schedule()` and reacquire them after or react to other events.

The function `inotify_read()` in `fs/notify/inotify/inotify_user.c`, which handles reading from the inotify file descriptor, is a straightforward example of using wait queues:

```
static ssize_t inotify_read(struct file *file, char __user *buf,
                          size_t count, loff_t *pos)
{
    struct fsnotify_group *group;
    struct fsnotify_event *kevent;
    char __user *start;
    int ret;
    DEFINE_WAIT(wait);

    start = buf;
    group = file->private_data;

    while (1) {
        prepare_to_wait(&group->notification_waitq,
                       &wait,
                       TASK_INTERRUPTIBLE);

        mutex_lock(&group->notification_mutex);
        kevent = get_one_event(group, count);
        mutex_unlock(&group->notification_mutex);

        if (kevent) {
            ret = PTR_ERR(kevent);
            if (IS_ERR(kevent))
                break;
            ret = copy_event_to_user(group, kevent, buf);
            fsnotify_put_event(kevent);
            if (ret < 0)
                break;
            buf += ret;
            count -= ret;
            continue;
        }

        ret = -EAGAIN;
        if (file->f_flags & O_NONBLOCK)
            break;
        ret = -EINTR;
        if (signal_pending(current))
            break;

        if (start != buf)
            break;

        schedule();
    }
}
```

```

    }
    finish_wait(&group->notification_waitq, &wait);

    if (start != buf && ret != -EFAULT)
        ret = buf - start;
    return ret;
}

```

This function follows the pattern laid out in our example. The main difference is that it checks for the condition in the body of the `while()` loop, instead of in the `while()` statement itself. This is because checking the condition is complicated and requires grabbing locks. The loop is terminated via `break`.

Waking Up

Waking is handled via `wake_up()`, which wakes up all the tasks waiting on the given wait queue. It calls `try_to_wake_up()`, which sets the task's state to `TASK_RUNNING`, calls `enqueue_task()` to add the task to the red-black tree, and sets `need_resched` if the awakened task's priority is higher than the priority of the current task. The code that causes the event to occur typically calls `wake_up()` itself. For example, when data arrives from the hard disk, the VFS calls `wake_up()` on the wait queue that holds the processes waiting for the data.

An important note about sleeping is that there are spurious wake-ups. Just because a task is awakened does not mean that the event for which the task is waiting has occurred; sleeping should always be handled in a loop that ensures that the condition for which the task is waiting has indeed occurred. Figure 4.1 depicts the relationship between each scheduler state.

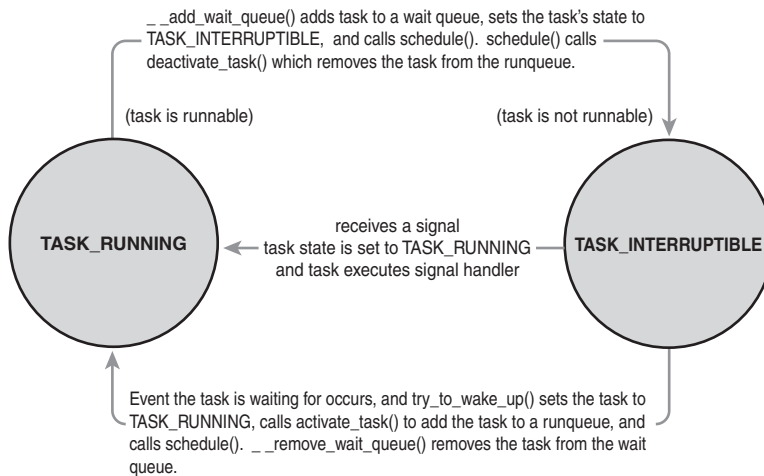


Figure 4.1 Sleeping and waking up.

Preemption and Context Switching

Context switching, the switching from one runnable task to another, is handled by the `context_switch()` function defined in `kernel/sched.c`. It is called by `schedule()` when a new process has been selected to run. It does two basic jobs:

- Calls `switch_mm()`, which is declared in `<asm/mmu_context.h>`, to switch the virtual memory mapping from the previous process's to that of the new process.
- Calls `switch_to()`, declared in `<asm/system.h>`, to switch the processor state from the previous process's to the current's. This involves saving and restoring stack information and the processor registers and any other architecture-specific state that must be managed and restored on a per-process basis.

The kernel, however, must know when to call `schedule()`. If it called `schedule()` only when code explicitly did so, user-space programs could run indefinitely. Instead, the kernel provides the `need_resched` flag to signify whether a reschedule should be performed (see Table 4.1). This flag is set by `scheduler_tick()` when a process should be preempted, and by `try_to_wake_up()` when a process that has a higher priority than the currently running process is awakened. The kernel checks the flag, sees that it is set, and calls `schedule()` to switch to a new process. The flag is a message to the kernel that the scheduler should be invoked as soon as possible because another process deserves to run.

Table 4.1 Functions for Accessing and Manipulating `need_resched`

Function	Purpose
<code>set_tsk_need_resched()</code>	Set the <code>need_resched</code> flag in the given process.
<code>clear_tsk_need_resched()</code>	Clear the <code>need_resched</code> flag in the given process.
<code>need_resched()</code>	Test the value of the <code>need_resched</code> flag; return true if set and false otherwise.

Upon returning to user-space or returning from an interrupt, the `need_resched` flag is checked. If it is set, the kernel invokes the scheduler before continuing.

The flag is per-process, and not simply global, because it is faster to access a value in the process descriptor (because of the speed of `current` and high probability of it being cache hot) than a global variable. Historically, the flag was global before the 2.2 kernel. In 2.2 and 2.4, the flag was an `int` inside the `task_struct`. In 2.6, it was moved into a single bit of a special flag variable inside the `thread_info` structure.

User Preemption

User preemption occurs when the kernel is about to return to user-space, `need_resched` is set, and therefore, the scheduler is invoked. If the kernel is returning to user-space, it

knows it is in a safe quiescent state. In other words, if it is safe to continue executing the current task, it is also safe to pick a new task to execute. Consequently, whenever the kernel is preparing to return to user-space either on return from an interrupt or after a system call, the value of `need_resched` is checked. If it is set, the scheduler is invoked to select a new (more fit) process to execute. Both the return paths for return from interrupt and return from system call are architecture-dependent and typically implemented in assembly in `entry.S` (which, aside from kernel entry code, also contains kernel exit code).

In short, user preemption can occur

- When returning to user-space from a system call
- When returning to user-space from an interrupt handler

Kernel Preemption

The Linux kernel, unlike most other Unix variants and many other operating systems, is a fully preemptive kernel. In nonpreemptive kernels, kernel code runs until completion. That is, the scheduler cannot reschedule a task while it is in the kernel—kernel code is scheduled cooperatively, not preemptively. Kernel code runs until it finishes (returns to user-space) or explicitly blocks. In the 2.6 kernel, however, the Linux kernel became preemptive: It is now possible to preempt a task at any point, so long as the kernel is in a state in which it is safe to reschedule.

So when is it safe to reschedule? The kernel can preempt a task running in the kernel so long as it does not hold a lock. That is, locks are used as markers of regions of nonpreemptibility. Because the kernel is SMP-safe, if a lock is not held, the current code is reentrant and capable of being preempted.

The first change in supporting kernel preemption was the addition of a preemption counter, `preempt_count`, to each process's `thread_info`. This counter begins at zero and increments once for each lock that is acquired and decrements once for each lock that is released. When the counter is zero, the kernel is preemptible. Upon return from interrupt, if returning to kernel-space, the kernel checks the values of `need_resched` and `preempt_count`. If `need_resched` is set and `preempt_count` is zero, then a more important task is runnable, and it is safe to preempt. Thus, the scheduler is invoked. If `preempt_count` is nonzero, a lock is held, and it is unsafe to reschedule. In that case, the interrupt returns as usual to the currently executing task. When all the locks that the current task is holding are released, `preempt_count` returns to zero. At that time, the unlock code checks whether `need_resched` is set. If so, the scheduler is invoked. Enabling and disabling kernel preemption is sometimes required in kernel code and is discussed in Chapter 9.

Kernel preemption can also occur explicitly, when a task in the kernel blocks or explicitly calls `schedule()`. This form of kernel preemption has always been supported because no additional logic is required to ensure that the kernel is in a state that is safe to

preempt. It is assumed that the code that explicitly calls `schedule()` knows it is safe to reschedule.

Kernel preemption can occur

- When an interrupt handler exits, before returning to kernel-space
- When kernel code becomes preemptible again
- If a task in the kernel explicitly calls `schedule()`
- If a task in the kernel blocks (which results in a call to `schedule()`)

Real-Time Scheduling Policies

Linux provides two real-time scheduling policies, `SCHED_FIFO` and `SCHED_RR`. The normal, not real-time scheduling policy is `SCHED_NORMAL`. Via the *scheduling classes* framework, these real-time policies are managed not by the Completely Fair Scheduler, but by a special real-time scheduler, defined in `kernel/sched_rt.c`. The rest of this section discusses the real-time scheduling policies and algorithm.

`SCHED_FIFO` implements a simple first-in, first-out scheduling algorithm without timeslices. A runnable `SCHED_FIFO` task is always scheduled over any `SCHED_NORMAL` tasks. When a `SCHED_FIFO` task becomes runnable, it continues to run until it blocks or explicitly yields the processor; it has no timeslice and can run indefinitely. Only a higher priority `SCHED_FIFO` or `SCHED_RR` task can preempt a `SCHED_FIFO` task. Two or more `SCHED_FIFO` tasks at the same priority run round-robin, but again only yielding the processor when they explicitly choose to do so. If a `SCHED_FIFO` task is runnable, all tasks at a lower priority cannot run until it becomes unrunnable.

`SCHED_RR` is identical to `SCHED_FIFO` except that each process can run only until it exhausts a predetermined timeslice. That is, `SCHED_RR` is `SCHED_FIFO` with timeslices—it is a real-time, round-robin scheduling algorithm. When a `SCHED_RR` task exhausts its timeslice, any other real-time processes at its priority are scheduled round-robin. The timeslice is used to allow only rescheduling of same-priority processes. As with `SCHED_FIFO`, a higher-priority process always immediately preempts a lower-priority one, and a lower-priority process can never preempt a `SCHED_RR` task, even if its timeslice is exhausted.

Both real-time scheduling policies implement static priorities. The kernel does not calculate dynamic priority values for real-time tasks. This ensures that a real-time process at a given priority *always* preempts a process at a lower priority.

The real-time scheduling policies in Linux provide soft real-time behavior. *Soft real-time* refers to the notion that the kernel tries to schedule applications within timing deadlines, but the kernel does not promise to always achieve these goals. Conversely, *hard real-time* systems are guaranteed to meet any scheduling requirements within certain limits. Linux makes no guarantees on the capability to schedule real-time tasks. Despite not having a design that guarantees hard real-time behavior, the real-time scheduling performance in Linux is quite good. The 2.6 Linux kernel is capable of meeting stringent timing requirements.

Real-time priorities range inclusively from zero to `MAX_RT_PRIO` minus 1. By default, `MAX_RT_PRIO` is 100—therefore, the default real-time priority range is zero to 99. This priority space is shared with the nice values of `SCHED_NORMAL` tasks: They use the space from `MAX_RT_PRIO` to $(\text{MAX_RT_PRIO} + 40)$. By default, this means the -20 to $+19$ nice range maps directly onto the priority space from 100 to 139.

Scheduler-Related System Calls

Linux provides a family of system calls for the management of scheduler parameters. These system calls allow manipulation of process priority, scheduling policy, and processor affinity, as well as provide an explicit mechanism to *yield* the processor to other tasks.

Various books—and your friendly system man pages—provide reference to these system calls (which are all implemented in the C library without much wrapper—they just invoke the system call). Table 4.2 lists the system calls and provides a brief description. How system calls are implemented in the kernel is discussed in Chapter 5, “System Calls.”

Table 4.2 Scheduler-Related System Calls

System Call	Description
<code>nice()</code>	Sets a process's nice value
<code>sched_setscheduler()</code>	Sets a process's scheduling policy
<code>sched_getscheduler()</code>	Gets a process's scheduling policy
<code>sched_setparam()</code>	Sets a process's real-time priority
<code>sched_getparam()</code>	Gets a process's real-time priority
<code>sched_get_priority_max()</code>	Gets the maximum real-time priority
<code>sched_get_priority_min()</code>	Gets the minimum real-time priority
<code>sched_rr_get_interval()</code>	Gets a process's timeslice value
<code>sched_setaffinity()</code>	Sets a process's processor affinity
<code>sched_getaffinity()</code>	Gets a process's processor affinity
<code>sched_yield()</code>	Temporarily yields the processor

Scheduling Policy and Priority-Related System Calls

The `sched_setscheduler()` and `sched_getscheduler()` system calls set and get a given process's scheduling policy and real-time priority, respectively. Their implementation, like most system calls, involves a lot of argument checking, setup, and cleanup. The important work, however, is merely to read or write the `policy` and `rt_priority` values in the process's `task_struct`.

The `sched_setparam()` and `sched_getparam()` system calls set and get a process's real-time priority. These calls merely encode `rt_priority` in a special `sched_param` structure. The calls `sched_get_priority_max()` and `sched_get_priority_min()` return the maximum and minimum priorities, respectively, for a given scheduling policy. The maximum priority for the real-time policies is `MAX_USER_RT_PRIO` minus one; the minimum is one.

For normal tasks, the `nice()` function increments the given process's static priority by the given amount. Only root can provide a negative value, thereby lowering the nice value and increasing the priority. The `nice()` function calls the kernel's `set_user_nice()` function, which sets the `static_prio` and `prio` values in the task's `task_struct` as appropriate.

Processor Affinity System Calls

The Linux scheduler enforces hard processor affinity. That is, although it tries to provide soft or natural affinity by attempting to keep processes on the same processor, the scheduler also enables a user to say, "This task must remain on this subset of the available processors no matter what." This hard affinity is stored as a bitmask in the task's `task_struct` as `cpus_allowed`. The bitmask contains one bit per possible processor on the system. By default, all bits are set and, therefore, a process is potentially runnable on any processor. The user, however, via `sched_setaffinity()`, can provide a different bitmask of any combination of one or more bits. Likewise, the call `sched_getaffinity()` returns the current `cpus_allowed` bitmask.

The kernel enforces hard affinity in a simple manner. First, when a process is initially created, it inherits its parent's affinity mask. Because the parent is running on an allowed processor, the child thus runs on an allowed processor. Second, when a processor's affinity is changed, the kernel uses the *migration threads* to push the task onto a legal processor. Finally, the load balancer pulls tasks to only an allowed processor. Therefore, a process only ever runs on a processor whose bit is set in the `cpus_allowed` field of its process descriptor.

Yielding Processor Time

Linux provides the `sched_yield()` system call as a mechanism for a process to explicitly yield the processor to other waiting processes. It works by removing the process from the active array (where it currently is, because it is running) and inserting it into the expired array. This has the effect of not only preempting the process and putting it at the end of its priority list, but also putting it on the expired list—guaranteeing it will not run for a

while. Because real-time tasks never expire, they are a special case. Therefore, they are merely moved to the end of their priority list (and not inserted into the expired array). In earlier versions of Linux, the semantics of the `sched_yield()` call were quite different; at best, the task was moved only to the end of its priority list. The yielding was often not for a long time. Nowadays, applications and even kernel code should be certain they truly want to give up the processor before calling `sched_yield()`.

Kernel code, as a convenience, can call `yield()`, which ensures that the task's state is `TASK_RUNNING` and then call `sched_yield()`. User-space applications use the `sched_yield()` system call.

Conclusion

The process scheduler is an important part of any kernel because running processes is (for most of us, at least) the point of using the computer in the first place. Juggling the demands of process scheduling is nontrivial, however: A large number of runnable processes, scalability concerns, trade-offs between latency and throughput, and the demands of various workloads make a one-size-fits-all algorithm hard to achieve. The Linux kernel's new CFS process scheduler, however, comes close to appeasing all parties and providing an optimal solution for most use cases with good scalability through a novel, interesting approach.

The previous chapter covered process management. This chapter ruminated on the theory behind process scheduling and the specific implementation, algorithms, and interfaces used by the current Linux kernel. The next chapter covers the primary interface that the kernel provides to running processes: system calls.

This page intentionally left blank

System Calls

In any modern operating system, the kernel provides a set of interfaces by which processes running in user-space can interact with the system. These interfaces give applications controlled access to hardware, a mechanism with which to create new processes and communicate with existing ones, and the capability to request other operating system resources. The interfaces act as the messengers between applications and the kernel, with the applications issuing various requests and the kernel fulfilling them (or returning an error). The existence of these interfaces, and the fact that applications are not free to directly do whatever they want, is key to providing a stable system.

Communicating with the Kernel

System calls provide a layer between the hardware and user-space processes. This layer serves three primary purposes. First, it provides an abstracted hardware interface for user-space. When reading or writing from a file, for example, applications are not concerned with the type of disk, media, or even the type of filesystem on which the file resides. Second, system calls ensure system security and stability. With the kernel acting as a middleman between system resources and user-space, the kernel can arbitrate access based on permissions, users, and other criteria. For example, this arbitration prevents applications from incorrectly using hardware, stealing other processes' resources, or otherwise doing harm to the system. Finally, a single common layer between user-space and the rest of the system allows for the virtualized system provided to processes, discussed in Chapter 3, "Process Management." If applications were free to access system resources without the kernel's knowledge, it would be nearly impossible to implement multitasking and virtual memory, and certainly impossible to do so with stability and security. In Linux, system calls are the only means user-space has of interfacing with the kernel; they are the only legal entry point into the kernel other than exceptions and traps. Indeed, other interfaces, such as device files or `/proc`, are ultimately accessed via system calls. Interestingly, Linux

implements far fewer system calls than most systems.¹ This chapter addresses the role and implementation of system calls in Linux.

APIs, POSIX, and the C Library

Typically, applications are programmed against an Application Programming Interface (API) implemented in user-space, not directly to system calls. This is important because no direct correlation is needed between the interfaces that applications make use of and the actual interface provided by the kernel. An API defines a set of programming interfaces used by applications. Those interfaces can be implemented as a system call, implemented through multiple system calls, or implemented without the use of system calls at all. The same API can exist on multiple systems and provide the same interface to applications while the implementation of the API itself can differ greatly from system to system. See Figure 5.1 for an example of the relationship between a POSIX API, the C library, and system calls.

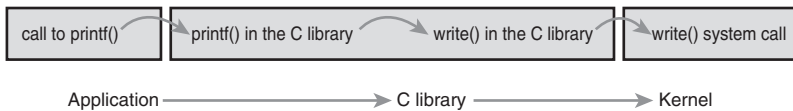


Figure 5.1 The relationship between applications, the C library, and the kernel with a call to **printf()**.

One of the more common application programming interfaces in the Unix world is based on the POSIX standard. Technically, POSIX is composed of a series of standards from the IEEE² that aim to provide a portable operating system standard roughly based on Unix. Linux strives to be POSIX- and SUSv3-compliant where applicable.

POSIX is an excellent example of the relationship between APIs and system calls. On most Unix systems, the POSIX-defined API calls have a strong correlation to the system calls. Indeed, the POSIX standard was created to resemble the interfaces provided by earlier Unix systems. On the other hand, some systems that are rather un-Unix, such as Microsoft Windows, offer POSIX-compatible libraries.

¹ There are about 335 system calls on x86. (Each architecture is allowed to define unique system calls.) Although not all operating systems publish their exact system calls, some operating systems are estimated to have more than one thousand. In the previous edition of this book, x86 had only 250 system calls.

² IEEE (eye-triple-E) is the Institute of Electrical and Electronics Engineers. It is a nonprofit professional association involved in numerous technical areas and responsible for many important standards, such as POSIX. For more information, visit <http://www.ieee.org>.

The system call interface in Linux, as with most Unix systems, is provided in part by the C library. The C library implements the main API on Unix systems, including the standard C library and the system call interface. The C library is used by all C programs and, because of C's nature, is easily wrapped by other programming languages for use in their programs. The C library additionally provides the majority of the POSIX API.

From the application programmer's point of view, system calls are irrelevant; all the programmer is concerned with is the API. Conversely, the kernel is concerned only with the system calls; what library calls and applications make use of the system calls is not of the kernel's concern. Nonetheless, it is important for the kernel to keep track of the potential uses of a system call and keep the system call as general and flexible as possible.

A meme related to interfaces in Unix is "Provide mechanism, not policy." In other words, Unix system calls exist to provide a specific function in an abstract sense. The manner in which the function is used is not any of the kernel's business.

Syscalls

System calls (often called *syscalls* in Linux) are typically accessed via function calls defined in the C library. They can define zero, one, or more arguments (inputs) and might result in one or more side effects,³ for example writing to a file or copying some data into a provided pointer. System calls also provide a return value of type `long`⁴ that signifies success or error—usually, although not always, a negative return value denotes an error. A return value of zero is usually (but again not always) a sign of success. The C library, when a system call returns an error, writes a special error code into the global `errno` variable. This variable can be translated into human-readable errors via library functions such as `perror()`.

Finally, system calls have a defined behavior. For example, the system call `getpid()` is defined to return an integer that is the current process's PID. The implementation of this syscall in the kernel is simple:

```
SYSCALL_DEFINE0(getpid)
{
    return task_tgid_vnr(current); // returns current->tgid
}
```

Note that the definition says nothing of the implementation. The kernel must provide the intended behavior of the system call but is free to do so with whatever implementation

³ Note the "might" here. Although nearly all system calls have a side effect (that is, they result in some change of the system's state), a few syscalls, such as `getpid()`, merely return some data from the kernel.

⁴ The use of type `long` is for compatibility with 64-bit architectures.

it wants as long as the result is correct. Of course, this system call is as simple as they come, and there are not too many other ways to implement it.⁵

`SYSCALL_DEFINE0` is simply a macro that defines a system call with no parameters (hence the `0`). The expanded code looks like this:

```
asmlinkage long sys_getpid(void)
```

Let's look at how system calls are defined. First, note the `asmlinkage` modifier on the function definition. This is a directive to tell the compiler to look only on the stack for this function's arguments. This is a required modifier for all system calls. Second, the function returns a `long`. For compatibility between 32- and 64-bit systems, system calls defined to return an `int` in user-space return a `long` in the kernel. Third, note that the `getpid()` system call is defined as `sys_getpid()` in the kernel. This is the naming convention taken with all system calls in Linux: System call `bar()` is implemented in the kernel as function `sys_bar()`.

System Call Numbers

In Linux, each system call is assigned a *syscall number*. This is a unique number that is used to reference a specific system call. When a user-space process executes a system call, the syscall number identifies which syscall was executed; the process does not refer to the syscall by name.

The syscall number is important; when assigned, it cannot change, or compiled applications will break. Likewise, if a system call is removed, its system call number cannot be recycled, or previously compiled code would aim to invoke one system call but would in reality invoke another. Linux provides a “not implemented” system call, `sys_ni_syscall()`, which does nothing except return `-ENOSYS`, the error corresponding to an invalid system call. This function is used to “plug the hole” in the rare event that a syscall is removed or otherwise made unavailable.

The kernel keeps a list of all registered system calls in the system call table, stored in `sys_call_table`. This table is architecture-specific; on x86-64 it is defined in `arch/i386/kernel/syscall_64.c`. This table assigns each valid syscall to a unique syscall number.

System Call Performance

System calls in Linux are faster than in many other operating systems. This is partly because of Linux's fast context switch times; entering and exiting the kernel is a streamlined and simple affair. The other factor is the simplicity of the system call handler and the individual system calls themselves.

⁵ You might be wondering why does `getpid()` return `tgid`, the thread group ID? In normal processes, the `TGID` is equal to the `PID`. With threads, the `TGID` is the same for all threads in a thread group. This enables the threads to call `getpid()` and get the same `PID`.

System Call Handler

It is not possible for user-space applications to execute kernel code directly. They cannot simply make a function call to a method existing in kernel-space because the kernel exists in a protected memory space. If applications could directly read and write to the kernel's address space, system security and stability would be nonexistent.

Instead, user-space applications must somehow signal to the kernel that they want to execute a system call and have the system switch to kernel mode, where the system call can be executed in kernel-space by the kernel on behalf of the application.

The mechanism to signal the kernel is a software interrupt: Incur an exception, and the system will switch to kernel mode and execute the exception handler. The exception handler, in this case, is actually the system call handler. The defined software interrupt on x86 is interrupt number 128, which is incurred via the `int $0x80` instruction. It triggers a switch to kernel mode and the execution of exception vector 128, which is the system call handler. The system call handler is the aptly named function `system_call()`. It is architecture-dependent; on x86-64 it is implemented in assembly in `entry_64.S`.⁶ Recently, x86 processors added a feature known as *sysenter*. This feature provides a faster, more specialized way of trapping into a kernel to execute a system call than using the `int` interrupt instruction. Support for this feature was quickly added to the kernel. Regardless of how the system call handler is invoked, however, the important notion is that somehow user-space causes an exception or trap to enter the kernel.

Denoting the Correct System Call

Simply entering kernel-space alone is not sufficient because multiple system calls exist, all of which enter the kernel in the same manner. Thus, the system call number must be passed into the kernel. On x86, the `syscall` number is fed to the kernel via the `eax` register. Before causing the trap into the kernel, user-space sticks in `eax` the number corresponding to the desired system call. The system call handler then reads the value from `eax`. Other architectures do something similar.

The `system_call()` function checks the validity of the given system call number by comparing it to `NR_syscalls`. If it is larger than or equal to `NR_syscalls`, the function returns `-ENOSYS`. Otherwise, the specified system call is invoked:

```
call *sys_call_table(,%rax,8)
```

Because each element in the system call table is 64 bits (8 bytes), the kernel multiplies the given system call number by four to arrive at its location in the system call table. On x86-32, the code is similar, with the 8 replaced by 4. See Figure 5.2.

⁶ Much of the following description of the system call handler is based on the x86 version. They are all similar.

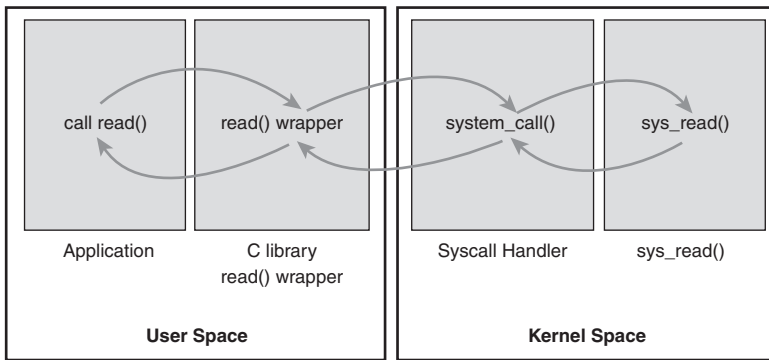


Figure 5.2 Invoking the system call handler and executing a system call.

Parameter Passing

In addition to the system call number, most syscalls require that one or more parameters be passed to them. Somehow, user-space must relay the parameters to the kernel during the trap. The easiest way to do this is via the same means that the syscall number is passed: The parameters are stored in registers. On x86-32, the registers `ebx`, `ecx`, `edx`, `esi`, and `edi` contain, in order, the first five arguments. In the unlikely case of six or more arguments, a single register is used to hold a pointer to user-space where all the parameters are stored.

The return value is sent to user-space also via register. On x86, it is written into the `eax` register.

System Call Implementation

The actual implementation of a system call in Linux does not need to be concerned with the behavior of the system call handler. Thus, adding a new system call to Linux is relatively easy. The hard work lies in designing and implementing the system call; registering it with the kernel is simple. Let's look at the steps involved in writing a new system call for Linux.

Implementing System Calls

The first step in implementing a system call is defining its purpose. What will it do? The syscall should have exactly one purpose. Multiplexing syscalls (a single system call that does wildly different things depending on a flag argument) is discouraged in Linux. Look at `ioctl()` as an example of what *not* to do.

What are the new system call's arguments, return value, and error codes? The system call should have a clean and simple interface with the smallest number of arguments possible. The semantics and behavior of a system call are important; they must not change, because existing applications will come to rely on them. Be forward thinking; consider

how the function might change over time. Can new functionality be added to your system call or will any change require an entirely new function? Can you easily fix bugs without breaking backward compatibility? Many system calls provide a flag argument to address forward compatibility. The flag is not used to multiplex different behavior across a single system call—as mentioned, that is not acceptable—but to enable new functionality and options without breaking backward compatibility or needing to add a new system call.

Designing the interface with an eye toward the future is important. Are you needlessly limiting the function? Design the system call to be as general as possible. Do not assume its use today will be the same as its use tomorrow. The *purpose* of the system call will remain constant but its *uses* may change. Is the system call portable? Do not make assumptions about an architecture’s word size or endianness. Chapter 19, “Portability,” discusses these issues. Make sure you are not making poor assumptions that will break the system call in the future. Remember the Unix motto: “Provide mechanism, not policy.”

When you write a system call, you need to realize the need for portability and robustness, not just today but in the future. The basic Unix system calls have survived this test of time; most of them are just as useful and applicable today as they were 30 years ago!

Verifying the Parameters

System calls must carefully verify all their parameters to ensure that they are valid and legal. The system call runs in kernel-space, and if the user can pass invalid input into the kernel without restraint, the system’s security and stability can suffer.

For example, file I/O syscalls must check whether the file descriptor is valid. Process-related functions must check whether the provided PID is valid. Every parameter must be checked to ensure it is not just valid and legal, but correct. Processes must not ask the kernel to access resources to which the process does not have access.

One of the most important checks is the validity of any pointers that the user provides. Imagine if a process could pass any pointer into the kernel, unchecked, with warts and all, even passing a pointer to which it did not have read access! Processes could then trick the kernel into copying data for which they did not have access permission, such as data belonging to another process or data mapped unreadable. Before following a pointer into user-space, the system must ensure that

- The pointer points to a region of memory in user-space. Processes must not be able to trick the kernel into reading data in kernel-space on their behalf.
- The pointer points to a region of memory in the process’s address space. The process must not be able to trick the kernel into reading someone else’s data.
- If reading, the memory is marked readable. If writing, the memory is marked writable. If executing, the memory is marked executable. The process must not be able to bypass memory access restrictions.

The kernel provides two methods for performing the requisite checks and the desired copy to and from user-space. Note kernel code must never blindly follow a pointer into user-space! One of these two methods must always be used.

For writing into user-space, the method `copy_to_user()` is provided. It takes three parameters. The first is the destination memory address in the process's address space. The second is the source pointer in kernel-space. Finally, the third argument is the size in bytes of the data to copy.

For reading from user-space, the method `copy_from_user()` is analogous to `copy_to_user()`. The function reads from the second parameter into the first parameter the number of bytes specified in the third parameter.

Both of these functions return the number of bytes they failed to copy on error. On success, they return zero. It is standard for the syscall to return `-EFAULT` in the case of such an error.

Let's consider an example system call that uses both `copy_from_user()` and `copy_to_user()`. This syscall, `silly_copy()`, is utterly worthless; it copies data from its first parameter into its second. This is suboptimal in that it involves an intermediate and extraneous copy into kernel-space for no gain. But it helps illustrate the point.

```
/*
 * silly_copy - pointless syscall that copies the len bytes from
 * 'src' to 'dst' using the kernel as an intermediary in the copy.
 * Intended as an example of copying to and from the kernel.
 */
SYSCALL_DEFINE3(silly_copy,
                unsigned long *, src,
                unsigned long *, dst,
                unsigned long len)
{
    unsigned long buf;

    /* copy src, which is in the user's address space, into buf */
    if (copy_from_user(&buf, src, len))
        return -EFAULT;

    /* copy buf into dst, which is in the user's address space */
    if (copy_to_user(dst, &buf, len))
        return -EFAULT;

    /* return amount of data copied */
    return len;
}
```

Both `copy_to_user()` and `copy_from_user()` may block. This occurs, for example, if the page containing the user data is not in physical memory but is swapped to disk. In that case, the process sleeps until the page fault handler can bring the page from the swap file on disk into physical memory.

A final possible check is for valid permission. In older versions of Linux, it was standard for syscalls that require *root* privilege to use `suser()`. This function merely checked

whether a user was root; this is now removed and a finer-grained “capabilities” system is in place. The new system enables specific access checks on specific resources. A call to `capable()` with a valid capabilities flag returns nonzero if the caller holds the specified capability and zero otherwise. For example, `capable(CAP_SYS_NICE)` checks whether the caller has the ability to modify nice values of other processes. By default, the superuser possesses all capabilities and nonroot possesses none. For example, here is the `reboot()` system call. Note how its first step is ensuring that the calling process has the `CAP_SYS_REBOOT`. If that one conditional statement were removed, any process could reboot the system.

```
SYSCALL_DEFINE4(reboot,
                int, magic1,
                int, magic2,
                unsigned int, cmd,
                void __user *, arg)
{
    char buffer[256];

    /* We only trust the superuser with rebooting the system. */
    if (!capable(CAP_SYS_BOOT))
        return -EPERM;

    /* For safety, we require "magic" arguments. */
    if (magic1 != LINUX_REBOOT_MAGIC1 ||
        (magic2 != LINUX_REBOOT_MAGIC2 &&
         magic2 != LINUX_REBOOT_MAGIC2A &&
         magic2 != LINUX_REBOOT_MAGIC2B &&
         magic2 != LINUX_REBOOT_MAGIC2C))
        return -EINVAL;

    /* Instead of trying to make the power_off code look like
     * halt when pm_power_off is not set do it the easy way.
     */
    if ((cmd == LINUX_REBOOT_CMD_POWER_OFF) && !pm_power_off)
        cmd = LINUX_REBOOT_CMD_HALT;

    lock_kernel();
    switch (cmd) {
    case LINUX_REBOOT_CMD_RESTART:
        kernel_restart(NULL);
        break;

    case LINUX_REBOOT_CMD_CAD_ON:
        C_A_D = 1;
        break;
```

```

case LINUX_REBOOT_CMD_CAD_OFF:
    C_A_D = 0;
    break;

case LINUX_REBOOT_CMD_HALT:
    kernel_halt();
    unlock_kernel();
    do_exit(0);
    break;

case LINUX_REBOOT_CMD_POWER_OFF:
    kernel_power_off();
    unlock_kernel();
    do_exit(0);
    break;

case LINUX_REBOOT_CMD_RESTART2:
    if (strncpy_from_user(&buffer[0], arg, sizeof(buffer) - 1) < 0) {
        unlock_kernel();
        return -EFAULT;
    }
    buffer[sizeof(buffer) - 1] = '\0';

    kernel_restart(buffer);
    break;

default:
    unlock_kernel();
    return -EINVAL;
}
unlock_kernel();
return 0;
}

```

See `<linux/capability.h>` for a list of all capabilities and what rights they entail.

System Call Context

As discussed in Chapter 3, the kernel is in process context during the execution of a system call. The current pointer points to the current task, which is the process that issued the syscall.

In process context, the kernel is capable of sleeping (for example, if the system call blocks on a call or explicitly calls `schedule()`) and is fully preemptible. These two points are important. First, the capability to sleep means that system calls can make use of the majority of the kernel's functionality. As we will see in Chapter 7, "Interrupts and

Interrupt Handlers,” the capability to sleep greatly simplifies kernel programming.⁷ The fact that process context is preemptible implies that, like user-space, the current task may be preempted by another task. Because the new task may then execute the same system call, care must be exercised to ensure that system calls are reentrant. Of course, this is the same concern that symmetrical multiprocessing introduces. Synchronizing reentrancy is covered in Chapter 9, “An Introduction to Kernel Synchronization,” and Chapter 10, “Kernel Synchronization Methods.”

When the system call returns, control continues in `system_call()`, which ultimately switches to user-space and continues the execution of the user process.

Final Steps in Binding a System Call

After the system call is written, it is trivial to register it as an official system call:

1. Add an entry to the end of the system call table. This needs to be done for each architecture that supports the system call (which, for most calls, is all the architectures). The position of the syscall in the table, starting at zero, is its system call number. For example, the tenth entry in the list is assigned syscall number nine.
2. For each supported architecture, define the syscall number in `<asm/unistd.h>`.
3. Compile the syscall into the kernel image (as opposed to compiling as a module). This can be as simple as putting the system call in a relevant file in `kernel/`, such as `sys.c`, which is home to miscellaneous system calls.

Look at these steps in more detail with a fictional system call, `foo()`. First, we want to add `sys_foo()` to the system call table. For most architectures, the table is located in `entry.S` and looks like this:

```
ENTRY(sys_call_table)
    .long sys_restart_syscall    /* 0 */
    .long sys_exit
    .long sys_fork
    .long sys_read
    .long sys_write
    .long sys_open                /* 5 */

    ...

    .long sys_eventfd2
    .long sys_epoll_create1
    .long sys_dup3                /* 330 */
```

⁷ Interrupt handlers cannot sleep and thus are much more limited in what they can do than system calls running in process context.

```

.long sys_pipe2
.long sys_inotify_init1
.long sys_preadv
.long sys_pwritev
.long sys_rt_tgsigqueueinfo    /* 335 */
.long sys_perf_event_open
.long sys_recvmmsg

```

The new system call is then appended to the tail of this list:

```

.long sys_foo

```

Although it is not explicitly specified, the system call is then given the next subsequent syscall number—in this case, 338. For each architecture you want to support, the system call must be added to the architecture's system call table. The system call does not need to receive the same syscall number under each architecture, as the system call number is part of the architecture's unique ABI. Usually, you would want to make the system call available to each architecture. Note the convention of placing the number in a comment every five entries; this makes it easy to find out which syscall is assigned which number.

Next, the system call number is added to `<asm/unistd.h>`, which currently looks somewhat like this:

```

/*
 * This file contains the system call numbers.
 */

#define __NR_restart_syscall    0
#define __NR_exit               1
#define __NR_fork               2
#define __NR_read               3
#define __NR_write              4
#define __NR_open               5

...

#define __NR_signalfd4          327
#define __NR_eventfd2           328
#define __NR_epoll_create1      329
#define __NR_dup3               330
#define __NR_pipe2              331
#define __NR_inotify_init1      332
#define __NR_preadv             333
#define __NR_pwritev            334
#define __NR_rt_tgsigqueueinfo  335
#define __NR_perf_event_open    336
#define __NR_recvmmsg           337

```

The following is then added to the end of the list:

```

#define __NR_foo                338

```

Finally, the actual `foo()` system call is implemented. Because the system call must be compiled into the core kernel image in all configurations, in this example we define it in `kernel/sys.c`. You should put it wherever the function is most relevant; for example, if the function is related to scheduling, you could define it in `kernel/sched.c`.

```
#include <asm/page.h>

/*
 * sys_foo - everyone's favorite system call.
 *
 * Returns the size of the per-process kernel stack.
 */
asmlinkage long sys_foo(void)
{
    return THREAD_SIZE;
}
```

That is it! Boot this kernel and user-space can invoke the `foo()` system call.

Accessing the System Call from User-Space

Generally, the C library provides support for system calls. User applications can pull in function prototypes from the standard headers and link with the C library to use your system call (or the library routine that, in turn, uses your syscall call). If you just wrote the system call, however, it is doubtful that glibc already supports it!

Thankfully, Linux provides a set of macros for wrapping access to system calls. It sets up the register contents and issues the trap instructions. These macros are named `__syscalln()`, where *n* is between 0 and 6. The number corresponds to the number of parameters passed into the syscall because the macro needs to know how many parameters to expect and, consequently, push into registers. For example, consider the system call `open()`, defined as

```
long open(const char *filename, int flags, int mode)
```

The syscall macro to use this system call without explicit library support would be

```
#define __NR_open 5
__syscall3(long, open, const char *, filename, int, flags, int, mode)
```

Then, the application can simply call `open()`.

For each macro, there are $2 + 2 \times n$ parameters. The first parameter corresponds to the return type of the syscall. The second is the name of the system call. Next follows the type and name for each parameter in order of the system call. The `__NR_open` define is in `<asm/unistd.h>`; it is the system call number. The `__syscall13` macro expands into a C function with inline assembly; the assembly performs the steps discussed in the previous section to push the system call number and parameters into the correct registers and issue

the software interrupt to trap into the kernel. Placing this macro in an application is all that is required to use the `open()` system call.

Let's write the macro to use our splendid new `foo()` system call and then write some test code to show off our efforts.

```
#define __NR_foo 283
__syscall0(long, foo)

int main ()
{
    long stack_size;

    stack_size = foo ();
    printf ("The kernel stack size is %ld\n", stack_size);

    return 0;
}
```

Why Not to Implement a System Call

The previous sections have shown that it is easy to implement a new system call, but that in no way should encourage you to do so. Indeed, you should exercise caution and restraint in adding new syscalls. Often, much more viable alternatives to providing a new system call are available. Let's look at the pros, cons, and alternatives.

The pros of implementing a new interface as a syscall are as follows:

- System calls are simple to implement and easy to use.
- System call performance on Linux is fast.

The cons:

- You need a syscall number, which needs to be officially assigned to you.
- After the system call is in a stable series kernel, it is written in stone. The interface cannot change without breaking user-space applications.
- Each architecture needs to separately register the system call and support it.
- System calls are not easily used from scripts and cannot be accessed directly from the filesystem.
- Because you need an assigned syscall number, it is hard to maintain and use a system call outside of the master kernel tree.
- For simple exchanges of information, a system call is overkill.

The alternatives:

- Implement a device node and `read()` and `write()` to it. Use `ioctl()` to manipulate specific settings or retrieve specific information.

- Certain interfaces, such as semaphores, can be represented as file descriptors and manipulated as such.
- Add the information as a file to the appropriate location in sysfs.

For many interfaces, system calls *are* the correct answer. Linux, however, has tried to avoid simply adding a system call to support each new abstraction that comes along. The result has been an incredibly clean system call layer with few regrets or deprecations (interfaces no longer used or supported). The slow rate of addition of new system calls is a sign that Linux is a relatively stable and feature-complete operating system.

Conclusion

In this chapter, we discussed what system calls are and how they relate to library calls and the application programming interface (API). We then looked at how the Linux kernel implements system calls and the chain of events required to execute a system call: trapping into the kernel, transmitting the syscall number and any arguments, executing the correct system call function, and returning to user-space with the syscall's return value.

We then went over how to add system calls and provided a simple example of using a new system call from user-space. The whole process was quite easy! As the simplicity of adding a new system call demonstrates, the work is all in the syscall's implementation. The rest of this book discusses concepts and kernel interfaces needed to write well-behaved, optimal, and safe system calls.

Finally, we wrapped up the chapter with a discussion on the pros and cons of implementing system calls and a brief list of the alternatives to adding new ones.

Patches, Hacking, and the Community

One of the greatest benefits of Linux is the large community of users and developers that surround it. The community provides eyes to check your code, experts to provide advice, and users to test and report issues. Most important, the community is the final arbiter of what code is accepted into Linux' official kernel tree. Understanding how the system works is extremely important.

The Community

If the Linux kernel community had to call somewhere home, it would be the *Linux Kernel Mailing List*. The Linux Kernel Mailing List (or as the regulars abbreviate it, just *lkml*) is the location of the majority of the announcements, discussions, debates, and flame wars over the kernel. New features are discussed, and most code is posted to the list before any action is taken. The list sees upward of 300 messages a day, so it is not for the faint of heart. Subscribing (or at least reading a digest or the archives) is recommended for anyone interested in serious kernel development. You can learn a lot simply by watching the wizards at work.

You can subscribe by sending the following message in plain text to `majordomo@vger.kernel.org`:

```
subscribe linux-kernel <your@email.address>
```

You can get more information at <http://vger.kernel.org/> and a FAQ is available at <http://www.tux.org/lkml/>.

Numerous websites and other mailing lists pertain to the kernel specifically and Linux in general. An excellent resource for beginning kernel hackers is <http://kernel-newbies.org/>—a website that, of all things, caters to those cutting their teeth on the kernel. Two other excellent sources of kernel information are <http://www.lwn.net/>, Linux Weekly News, which has a great kernel news section, and <http://www.kerneltrap.org/>, Kernel Trap, which provides insightful commentary on kernel development.

Linux Coding Style

The Linux Kernel, like any large software project, has a defined coding style that stipulates the formatting, style, and layout of your code. This is done not because the Linux kernel style is superior (although it might be) or because your style is illegible, but because *consistency* of coding style is crucial to *productivity* in coding. Yet it is often argued that coding style is irrelevant because it does not affect the compiled object code. In a large project, such as the kernel, in which many developers are involved, consistency of coding style is crucial. Consistency implies familiarity, which leads to ease of reading, lack of confusion, and further expectations that code will continue to follow a given style. This increases the number of developers who can read your code, and the amount of code in which you can read. In an open-source project, the more eyes the better.

It is not so important *what* style is chosen as long as one is indeed selected and used exclusively. Fortunately, Linus long ago laid out the style we should use and most code sticks to it. The majority of the style is covered in Linus's usual humor in the file `Documentation/CodingStyle` in the kernel source tree.

Indentation

The stylistic convention for indentation is to use tabs that are eight characters in length. This does not mean it is okay to use eight spaces for indentation. Each level of indentation is a tab over from the previous, and a tab is eight characters in length. For example:

```
static void get_new_ship(const char *name)
{
    if (!name)
        name = DEFAULT_SHIP_NAME;
    get_new_ship_with_name(name);
}
```

For unclear reasons, this rule is one of the most commonly broken, despite its high impact on readability. Eight-character tabs make clearly identifying indentation of different code blocks orders of magnitude easier after hours of hacking. The downside, of course, of eight character tabs is that after several levels of indentation, not much usable space is left on the line. This is compounded by 80-character line length limits (see subsequent section). Linus' rejoinder to this is that your code should not be so complex and convoluted as to require more than two or three levels of indentation. Need you go that deep, he argues, you should refactor your code to pull out layers of complexity (and thus levels of indentation) into separate functions.

Switch Statements

Subordinate `case` labels should be indented to the same level as the parent `switch` statement, which helps alleviate the impact of eight character tabs. For example:

```
switch (animal) {
```

```

case ANIMAL_CAT:
    handle_cats();
    break;
case ANIMAL_WOLF:
    handle_wolves();
    /* fall through */
case ANIMAL_DOG:
    handle_dogs();
    break;
default:
    printk(KERN_WARNING "Unknown animal %d!\n", animal);
}

```

It is common (and good) practice to comment when deliberately falling through from one case statement to another, as shown in this example.

Spacing

This section covers the spacing around symbols and keywords, not the spacing used in indentation, which we covered in the last two sections. Generally speaking, Linux coding style dictates spaces around most keywords and no spaces between functions and their parentheses. For example:

```

if (foo)
while (foo)
for (i = 0; i < NR_CPUS; i++)
switch (foo)

```

Conversely, functions, macros, and keywords that look like functions—such as `sizeof`, `typeof`, and `alignof`—similarly have no space between the keyword and the parenthesis.

```

wake_up_process(task);
size_t nlongs = BITS_TO_LONG(nbits);
int len = sizeof(struct task_struct);
typeof(*p)
__alignof__(struct sockaddr *)
__attribute__((packed))

```

Within parentheses, there is no space proceeding or preceding the argument, as previously shown. For example, this is verboten:

```

int prio = task_prio( task ); /* BAD STYLE! */

```

Around most binary and tertiary operators, put a space on either side of the operator. For example:

```

int sum = a + b;
int product = a * b;
int mod = a % b;
int ret = (bar) ? bar : 0;

```

```

return (ret ? 0 : size);
int nr = nr ? : 1; /* allowed shortcut, same as "nr ? nr : 1" */
if (x < y)
if (tsk->flags & PF_SUPERPRIV)
mask = POLLIN | POLLRDNORM;

```

Conversely, around most unary operators, put no space between the operator and the operand:

```

if (!foo)
int len = foo.len;
struct work_struct *work = &dwork->work;
foo++;
-bar;
unsigned long inverted = ~mask;

```

Getting the spacing right around the dereference operator is particularly important. The correct style is

```
char *strcpy(char *dest, const char *src)
```

Placing a space on either side of the dereference operator is incorrect style:

```
char * strcpy(char * dest, const char * src) /* BAD STYLE */
```

Also incorrect is the C++ style of placing the dereference operator next to the type:

```
char* strcpy(char* dest, const char* src) /* BAD STYLE */
```

Braces

Brace placement is personal, and few technical reasons exist for one convention over the other, but we have to agree on something. The accepted kernel style is to put the opening brace on the first line, at the end of the statement. The closing brace goes on a new line as the first character. Following is an example:

```

if (strcmp(buf, "NO_", 3) == 0) {
    neg = 1;
    cmp += 3;
}

```

If the following token is a continuation of the same statement, the closing brace is not on a line by itself, but on a line shared with that token. For example:

```

if (ret) {
    sysctl_sched_rt_period = old_period;
    sysctl_sched_rt_runtime = old_runtime;
} else {
    def_rt_bandwidth.rt_runtime = global_rt_runtime();
    def_rt_bandwidth.rt_period = ns_to_ktime(global_rt_period());
}

```

And this example:

```
do {
    percpu_counter_add(&ca->cpustat[idx], val);
    ca = ca->parent;
} while (ca);
```

This rule is broken for functions, because functions cannot nest inside functions:

```
unsigned long func(void)
{
    /* ... */
}
```

Finally, statements that do not *need* braces can omit them. For example, the following is encouraged but not required:

```
if (cnt > 63)
    cnt = 63;
```

The logic behind all this is *K&R*.¹ Most of Linux coding style follows *K&R Style*, which is the C coding style used in that famous book.

Line Length

Lines of source code should be kept to fewer than 80 characters in length. This allows code to fit lengthwise on a standard 80×24 terminal.

There is no accepted standard on what to do in cases where code absolutely must wrap 80 characters. Some developers just allow the line to wrap, letting their editor handle the chore of displaying the code in a readable fashion. Other developers break up the lines, manually inserting line breaks where appropriate, perhaps starting each new line a tab stop over from the original.

Similarly, some developers line up function parameters that wrap lines with the open parenthesis. For example:

```
static void get_new_parrot(const char *name,
                          unsigned long disposition,
                          unsigned long feather_quality)
```

Other developers break up the lines but do not line the parameters up, instead use a standard two tabs:

```
int find_pirate_flag_by_color(const char *color,
                             const char *name, int len)
```

¹ The C Programming Language, by Brian Kernighan and Dennis Ritchie (Prentice Hall, ISBN# 0-13-11-362-8), nicknamed *K&R*, is the bible of C, written by C's author and his colleague.

As there is no definitive rule in this case, the choice is left up to you, the developer. Many kernel contributors, including myself, prefer the former example: Manually break up lines greater than 80 characters in length, trying to align the resulting new lines cleanly with the previous line.

Naming

No name should employ *CamelCase*, *Studly Caps*, or other mixed case schemes. Calling a local variable `idx` or even just `i` is perfectly fine if it is clear what it does. A cute name such as `theLoopIndex` is unacceptable. Hungarian notation (encoding the variable type in the variable name) is unnecessary and should never be used. This is C, not Java; Unix, not Windows.

Nonetheless, global variables and functions should have descriptive names, in lowercase and delimited via an underscore as needed. Calling a global function `atty()` is confusing; a name such as `get_active_tty()` is much more acceptable. This is Linux, not BSD.

Functions

As a rule of thumb, functions should not exceed one or two screens of text and should have fewer than ten local variables. A function should do one thing and do it well. There is no harm in breaking a function into a series of smaller functions. If you are worried about function call overhead, employ *inline functions* via the `inline` keyword.

Comments

Commenting your code is important, but the commenting must be done correctly. Generally, you want to describe *what* and *why* your code is doing what it is doing, not *how* it is doing it. The *how* should be apparent from the code itself. If not, you might need to rethink and refactor what you wrote. Additionally, comments should not include who wrote a function, the modification date, or other trivial nonsense. Such information is generally acceptable at the top of the source file, however.

The kernel uses C-style comments, even though `gcc` supports C++-style comments, too. The general style of a comment in the kernel resembles:

```
/*
 * get_ship_speed() - return the current speed of the pirate ship
 * We need this to calculate the ship coordinates. As this function can sleep,
 * do not call while holding a spinlock.
 */
```

In comments, important notes are often prefixed with `"xxx:"`, and bugs are often prefixed with `"FIXME:"` like so:

```
/*
```

```
* FIXME: We assume dog == cat which may not be true in the future
*/
```

The kernel has a facility for self-generating documentation. It is based on GNOME-doc, but slightly modified and renamed Kernel-doc. To create the standalone documentation in HTML format, run

```
make htmldocs
```

Or for postscript

```
make psdocs
```

You can use the system to document your functions by following a special format for your comments:

```
/**
 * find_treasure - find 'X marks the spot'
 * @map - treasure map
 * @time - time the treasure was hidden
 *
 * Must call while holding the pirate_ship_lock.
 */
void find_treasure(int map, struct timeval *time)
{
    /* ... */
}
```

For more information, see `Documentation/kernel-doc-nano-HOWTO.txt`.

Typedefs

The Linux kernel developer community employs a strong dislike of the `typedef` operator. Their rationale is

- `typedef` hides the real type of data structures.
- Because the type is hidden, code is more prone to do bad things, such as pass a structure by value on the stack.
- `typedef` is just being lazy.

Therefore, to avoid ridicule, avoid `typedef`.

Of course, there are a few good uses of `typedefs`: hiding an architecture-specific implementation of a variable or providing forward compatibility when a type may change. Decide carefully whether the `typedef` is truly needed or exists just to reduce the number of characters you need to type.

Use Existing Routines

Do not reinvent the wheel. The kernel provides string manipulation functions, compression routines, and a linked list interface, so use them.

Do not wrap existing interfaces in generic interfaces. Often you see code that was obviously ported from one operating system to Linux, and various kernel interfaces are wrapped in some gross glue function. No one likes this, so just use the provided interfaces directly.

Minimize `ifdefs` in the Source

Putting `ifdef` preprocessor directives directly in the C source is frowned upon. You should never do something like the following in your functions:

```
...
#ifdef CONFIG_FOO
    foo();
#endif
...
```

Instead, define `foo()` to nothing if `CONFIG_FOO` is not set:

```
#ifdef CONFIG_FOO
static int foo(void)
{
    /* .. */
}
#else
static inline int foo(void) { }
#endif /* CONFIG_FOO */
```

Then, you can unconditionally call `foo()`. Let the compiler do the work for you.

Structure Initializers

Labeled identifiers need to be used to initialize structures. This is good because it prevents structure changes from resulting in incorrect initialization. It also enables values to be omitted. Unfortunately, C99 adopted quite an ugly format for labeled identifiers, and `gcc` is deprecating usage of the previous GNU-style labeled identifier, which was rather handsome. Consequently, kernel code needs to use the new C99 labeled identifier format, however ugly it is:

```
struct foo my_foo = {
    .a    = INITIAL_A,
    .b    = INITIAL_B,
};
```

In this code, `a` and `b` are members of `struct foo` and `INITIAL_A` and `INITIAL_B` are their initialized values, respectively. If a field is not set, it is set to its default value per ANSI C (for example, pointers are `NULL`, integers are zero, and floats are `0.0`). For exam-

ple, if `struct foo` also has `int c` as a member, the previous statement would initialize `c` to zero.

Fixing Up Code Ex Post Facto

If a pile of code falls into your lap that fails to even mildly resemble the Linux kernel coding style, do not fret. A little elbow grease and the `indent` utility can make everything perfect. `indent`, an excellent GNU utility found on most Linux systems, formats source according to given rules. The default settings are for the GNU coding style, which is not too pretty. To get the utility to follow the Linux kernel style, execute the following:

```
indent -kr -i8 -ts8 -sob -l80 -ss -bs -psl <file>
```

This instructs the utility to format the code according to the kernel coding style. Alternatively, the script `scripts/Lindent` automatically invokes `indent` with the desired options.

Chain of Command

Kernel hackers are the developers who work on the kernel. Some do it for pay, some as a hobby, but nearly all for fun. Kernel hackers with significant contributions are listed in the `CREDITS` file in the root of the kernel source tree.

Most parts of the kernel have an associated *maintainer*. The maintainer is the individual (or individuals) who is in charge of specific parts of the kernel. For example, each individual driver has an associated maintainer. Each kernel subsystem—for example, network-ing—also has an associated maintainer. The maintainer for a specific driver or subsystem is usually listed in the file `MAINTAINERS`, which is also located in the root of the kernel source tree.

There is a special type of maintainer, known as the kernel maintainer. This individual actually maintains the kernel tree. Historically, Linus maintains the development kernel (where the real fun is) and the stable kernel for some period after development ends. Shortly after a development kernel becomes a stable kernel, Linus passes the torch to one of the top kernel developers. That developer continues to maintain the tree while Linus begins work on the new development tree. Given the “new world order” in which development on 2.6 continues in perpetuity, Linus remains the maintainer of the 2.6 kernel series. Another developer maintains the 2.4 series, which is in a strict bug-fix-only mode.

Submitting Bug Reports

If you encounter a bug, the best course of action is to write a fix, create a patch, test it, and submit it as discussed in the following sections. Of course, you can also report the problem and get someone to fix it for you.

The most important part of submitting a bug report is fully describing the problem. Describe the symptoms, any system output, and a fully decoded oops (if there is an oops). More important, if you can, provide steps to reliably reproduce the problem and a brief description of your hardware.

Deciding to whom to send the bug report is the next step. The file `MAINTAINERS`, in the root of the kernel source tree, lists the individuals associated with each driver and subsystem—they should receive any issues related to the code they maintain. If you cannot find an interested party, send the report to the Linux Kernel Mailing List at `linux-kernel@vger.kernel.org`. Even if you do find a maintainer, CC the kernel mailing list.

The files `REPORTING-BUGS` and `Documentation/oops-tracing.txt` provide more information.

Patches

All changes to the Linux kernel are distributed in the form of patches, which are the output of the GNU `diff(1)` program in a form that is readable by the `patch(1)` program.

Generating Patches

The simplest way to generate a patch is to have two source trees, one that is the vanilla stock kernel and another that is the stock tree with your modifications. A common scheme is to name the stock tree `linux-x.y.z` (which is what the source tarball extracts to, initially) and to name your modified tree simply `linux`. Then, to generate a patch of the two trees, issue the following command from one directory below your trees:

```
diff -urN linux-x.y.z/ linux/ > my-patch
```

This is typically done somewhere in your home, and not `/usr/src/linux` so that you do not need to be root. The `-u` flag specifies that the unified diff format should be used. Without this, the patch is ugly and not readable by humans. The `-r` flag instructs `diff` to recursively diff all directories, and the `-N` flag specifies that new files in the modified tree should be included in the diff. Alternatively, if you need to diff only a single file, you can do

```
diff -u linux-x.y.z/some/file linux/some/file > my-patch
```

You need to always diff the trees from one directory below your source trees. This creates a patch that is usable by others, even if their directory names differ. To apply a patch made in this format, do the following from the root of your source tree:

```
patch -p1 < ../my-patch
```

In this example, the patch is named `my-patch` and is created one directory below the current. The `-p1` flag instructs `diff` to strip the first directory from the patch. This enables you to apply a patch regardless of the directory-naming convention used by the patch maker.

A useful utility is `diffstat`, which generates a histogram of a patch's changes (line additions and removals). To generate the output on one of your patches, do

```
diffstat -p1 my-patch
```

It is often useful to include this output when you post a patch to `lkml`. Because the `patch(1)` program ignores all lines until a diff is detected, you can even include a short description at the top of the patch.

Generating Patches with Git

If you use Git to manage your source tree, you need to use Git to likewise generate your patches—there is no point going through all the aforementioned manual steps *and* bear the complexity of Git. Generating patches with Git is an easy two-part process. First, you need to author and then locally *commit* your changes. Making changes to a Git tree is the same as a standard source tree. You do not need to do anything special to edit a file stored in Git. After you make your changes, you need to commit them to your Git repository:

```
git commit -a
```

The `-a` flag instructs Git to commit *all* your changes. If you only want to commit changes to a specific file, you can do that, too:

```
git commit some/file.c
```

Even with the `-a` flag, however, Git will not commit new files until they are explicitly added to the repository. To add a file and then commit it (and all other changes), issue the following two commands:

```
git add some/other/file.c
git commit -a
```

When you run *git commit*, Git enables you to enter a change log. Make this entry verbose and complete, fully explaining the commit. (We cover exactly what to include in the next section.) You can create multiple commits against your repository. Thanks to Git's design, subsequent commits can even be against the same file, building off of each other. When you have a commit (or two) in your tree, you can generate a patch for each commit, which you can treat as you do the patches described in the previous section:

```
git format-patch origin
```

This generates patches for all commits in your repository and not in the original tree. Git creates the patches in the root of your kernel source tree. To generate patches for only the last *N* commits, you can execute the following:

```
git format-patch -N
```

For example, this command generates a patch for only the last commit:

```
git format-patch -1
```

Submitting Patches

Patches should be generated as described in the previous section. If the patch touches a specific driver or subsystem, the patch should be sent to the maintainer listed in MAINTAINER. Either way, the Linux Kernel Mailing List at `linux-kernel@vger.kernel.org` should be carbon copied. The patch should be sent to the kernel maintainer (for example, Linus) only after extensive discussion, or if the patch is trivial and clearly correct.

Typically, the subject line of the email containing the patch is of the form “[PATCH] *brief description*.” The body of the email describes in technical detail the changes your patch makes and the rationale behind them. Be as specific as possible. Somewhere in the email, note the kernel version against which the patch was created.

Most kernel developers want to read your patch inline with your email and optionally save the whole thing to a single file. Consequently, it is best to insert the patch directly inline in the email, at the end of your message. Be aware that some email clients might wrap lines or otherwise change formatting; this breaks the patch and annoys developers. If your email client does this, see whether it has an “Insert Inline,” “Preformat,” or similar feature. Otherwise, attaching the patch as plain text without encoding works, too.

If your patch is large or contains several logical changes, you should break the patch into chunks, with each chunk representing a logical change. For example, if you both introduce a new API and change a handful of drivers to use it, you can break the changes into two patches (the new API and then the driver changeover) and two emails. If any chunk requires a previous patch, explicitly state that.

After posting, remain patient and wait for a reply. Do not be discouraged by any negative response—at least you got a response! Discuss the issues and provide updated patches as needed. If you fail to receive *any* response, try to discover what was wrong and resolve the issues. Solicit additional comments from the mailing list and maintainer. With luck, you might see your changes in a future kernel release—congratulations!

Conclusion

The most important quality of any hacker is desire and drive—an itch to scratch, and the determination to scratch it. This book provided a tour of key parts of the kernel, discussing interfaces, data structures, algorithms, and rationale. It provided an insider’s view of the kernel, in a practical fashion, to satisfy your curiosity or get you off the ground running in your kernel endeavors.

As I have said before, however, the only way to start is by *reading* and *writing* code. Linux provides a community that not only enables but also *encourages* both activities—so start reading and coding! Happy hacking!